

Praca Dyplomowa Inżynierska

Adam Kliś
217584

Projekt systemu autoryzacji w oparciu o funkcjonalność tokenów

Design of an authorization system based on token functionality

Praca dyplomowa na kierunku:
Informatyka

Praca wykonana pod kierunkiem
dr inż. Artur Krupa
Instytut Informatyki Technicznej
Katedra Sztucznej Inteligencji

Warszawa, rok 2026



SZKOŁA GŁÓWNA
GOSPODARSTWA
WIEJSKIEGO

Wydział Zastosowań
Informatyki
i Matematyki

Oświadczenie Promotora pracy

Oświadczam, że niniejsza praca została przygotowana pod moim kierunkiem i stwierdzam, że spełnia ona warunki do przedstawienia tej pracy w postępowaniu o nadanie tytułu zawodowego.

Data

Podpis promotora

Oświadczenie autora pracy

Świadom/a odpowiedzialności prawnej, w tym odpowiedzialności karnej za złożenie fałszywego oświadczenia, oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami prawa, w szczególności z ustawą z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz. U. 2019 poz. 1231 z późn. zm.).

Oświadczam, że przedstawiona praca nie była wcześniej podstawą żadnej procedury związanej z nadaniem dyplomu lub uzyskaniem tytułu zawodowego.

Oświadczam, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną. Przyjmuję do wiadomości, że praca dyplomowa poddana zostanie procedurze antyplagiatowej.

Data

Podpis autora pracy

Streszczenie

Projekt systemu autoryzacji w oparciu o funkcjonalność tokenów

Praca przedstawia projekt oraz implementację autorskiego systemu uwierzytelniania i autoryzacji w środowisku ASP.NET Core, opartą na paradygmacie Zero Trust. Głównym celem inżynierskim było stworzenie wydajnej alternatywy dla standardowych tokenów JWT. Zaimplementowane rozwiązanie wykorzystuje autorski algorytm kompresji ról użytkowników do postaci szesnastkowych masek bitowych, co pozwala na walidację uprawnień z wykorzystaniem natywnych operacji procesora w czasie stałym $O(1)$. Projekt obejmuje mechanizmy rotacji tokenów odświeżających, dynamiczne skanowanie atrybutów uprawnień za pomocą mechanizmu refleksji oraz wbudowane testy jednostkowe. Analiza wykazała zgodność architektury z wytycznymi bezpieczeństwa OWASP Top 10 oraz znaczną redukcję narzutu sieciowego w porównaniu ze standardowymi rozwiązaniami rynkowymi.

Słowa kluczowe – uwierzytelnianie, autoryzacja, token, ASP.NET Core, Zero Trust, kryptografia, maski bitowe

Summary

Design of an authorization system based on token functionality

This thesis presents the design and implementation of a custom authentication and authorization system in the ASP.NET Core environment, based on the Zero Trust paradigm. The main engineering objective was to create a high-performance alternative to standard JWT tokens. The developed solution utilizes a proprietary algorithm for compressing user roles into hexadecimal bitmasks, which allows for access validation using native processor operations in $O(1)$ constant time. The project incorporates refresh token rotation mechanisms, dynamic permission scanning via reflection, and built-in unit tests. The analysis demonstrated the architecture's compliance with OWASP Top 10 security guidelines and a significant reduction in network overhead compared to standard market solutions.

Keywords – authentication, authorization, token, ASP.NET Core, Zero Trust, cryptography, bitmasks

Słownik pojęć

- **API (Application Programming Interface)** – interfejs programistyczny aplikacji pozwalający na komunikację między systemami.
- **AuthN (Authentication)** – uwierzytelnianie; proces weryfikacji tożsamości użytkownika lub systemu (np. na podstawie loginu i hasła).
- **AuthZ (Authorization)** – autoryzacja; proces weryfikacji, czy dany (już uwierzytelniony) użytkownik posiada uprawnienia do wykonania określonej operacji.
- **CPU (Central Processing Unit)** – główny procesor komputera.
- **DI (Dependency Injection)** – wstrzykiwanie zależności; wzorzec projektowy polegający na przekazywaniu gotowych obiektów (zależności) do klas, które ich używają, zamiast tworzyć je wewnątrz tych klas.
- **DTO (Data Transfer Object)** – obiekt transferu danych; wzorzec projektowy służący do przenoszenia agregatów danych pomiędzy procesami lub warstwami aplikacji.
- **EF Core (Entity Framework Core)** – wieloplatformowe narzędzie ORM dla ekosystemu .NET rozwijane przez firmę Microsoft.
- **HMAC (Keyed-Hash Message Authentication Code)** – kryptograficzna funkcja mieszająca wykorzystująca klucz tajny do weryfikacji integralności i autentyczności wiadomości.
- **HTTP (Hypertext Transfer Protocol)** – bezstanowy protokół komunikacyjny będący fundamentem sieci Web.
- **IoC (Inversion of Control)** – odwrócenie sterowania; paradygmat programowania, w którym zewnętrzny kontener zarządza przepływem sterowania i tworzeniem obiektów.
- **JSON (JavaScript Object Notation)** – lekki, tekstowy format wymiany danych.
- **JWT (JSON Web Token)** – otwarty standard (RFC 7519) definiujący kompaktowy sposób bezpiecznego przesyłania informacji w formacie JSON między stronami.
- **LINQ (Language Integrated Query)** – zintegrowany z językiem C# mechanizm tworzenia zapytań do różnych źródeł danych.
- **MFA (Multi-Factor Authentication)** – uwierzytelnianie wieloskładnikowe; metoda weryfikacji tożsamości wymagająca użycia co najmniej dwóch niezależnych dowodów.
- **ORM (Object-Relational Mapping)** – mapowanie obiektowo-relacyjne; technika programistyczna wiążąca struktury obiektowe w kodzie z tabelami relacyjnej bazy danych.
- **RAM (Random Access Memory)** – główna pamięć operacyjna komputera.
- **SIEM (Security Information and Event Management)** – system zarządzania informacjami i zdarzeniami bezpieczeństwa, służący do monitorowania ruchu sieciowego.

go.

- **SQL JOIN** – operacja złączenia w relacyjnych bazach danych polegająca na logicznym łączeniu rekordów z dwóch lub więcej tabel.
- **Token** – cyfrowy nośnik informacji, często zabezpieczony kryptograficznie, reprezentujący tożsamość lub uprawnienia użytkownika.
- **TOTP (Time-based One-Time Password)** – algorytm generujący jednorazowe hasła oparte na aktualnym czasie.
- **Zero Trust** – model bezpieczeństwa zakładający, że żadne urządzenie ani użytkownik nie jest domyślnie zaufany, nawet jeśli znajduje się wewnątrz sieci korporacyjnej.

Spis treści

Słownik pojęć	7
1 Wstęp	12
2 Przegląd technologii i stan wiedzy	15
2.1 Uwierzytelnianie a autoryzacja	15
2.2 Protokół HTTP i problem braku stanu	15
2.3 Ewolucja: Od sesji stanowych do bezstanowych tokenów	16
2.4 Anatomia standardu JSON Web Token (JWT)	16
2.5 Paradygmat Inversion of Control i mechanizm DI	18
2.6 Środowisko uruchomieniowe i persystencja danych	18
3 Projekt i architektura systemu	21
3.1 Koncepcja autorskiego tokenu	21
3.2 Algorytm podpisu cyfrowego	22
3.3 Weryfikacja tokenu i integralność	22
3.4 Zarządzanie rolami przy użyciu masek bitowych	24
3.5 Analiza złożoności obliczeniowej i pamięciowej	25
3.6 Cykl życia sesji i rotacja tokenów odświeżających	25
3.7 Diagram klas: Warstwa danych	26
3.8 Diagram klas: Kontrakty i modele	27
3.9 Diagram klas: Logika i punkty dostępu	28
3.10 Diagram przypadków użycia	28
3.11 Diagram sekwencji weryfikacji uprawnień	29
4 Implementacja rozwiązania	31
4.1 Logika biznesowa i potok ASP.NET Core	31
4.2 Rejestracja i bezpieczne przechowywanie haseł	32
4.3 Proces logowania i weryfikacji tożsamości	33
4.4 Implementacja odświeżania sesji i rotacji tokenów	36
4.5 Konfiguracja i dynamiczne rozszerzanie ładunku (Custom Payload)	37
4.6 Automatyzacja zarządzania i migracja ról (Bootstrapper)	39
4.7 Dynamiczne skanowanie atrybutów	40
4.8 Proces bezpiecznej migracji uprawnień	40
4.9 Implementacja operacji kryptograficznych i ochrona przed atakami	42
4.10 Filtry potoku HTTP i weryfikacja tożsamości	44

4.11	Zaawansowana autoryzacja oparta o role	46
5	Wdrożenie biblioteki i analiza porównawcza	49
5.1	Strategie integracji warstwy danych	49
5.2	Proces wdrożenia krok po kroku	50
5.3	Porównanie ze standardem ASP.NET Core Identity i JWT	50
5.4	Analiza wydajnościowa oprogramowania	51
6	Podsumowanie i perspektywy rozwoju	53
	Bibliografia	55

1 Wstęp

Wraz z dynamicznym rozwojem aplikacji internetowych oraz popularyzacją architektury rozproszonej, mechanizmy zarządzania dostępem stały się jednym z najbardziej krytycznych elementów systemów informatycznych. Zapewnienie bezpieczeństwa danych, przy jednoczesnym zachowaniu wysokiej wydajności i skalowalności, stanowi wyzwanie, z którym inżynierowie oprogramowania mierzą się od początków istnienia sieci WWW. Jak wskazuje dokumentacja standardów kryptograficznych [12], ochrona informacji wymaga implementacji niezawodnych mechanizmów na poziomie samej aplikacji, a nie tylko infrastruktury sieciowej.

We wczesnych fazach rozwoju aplikacji internetowych serwery generowały statyczne strony HTML, a potrzeba śledzenia tożsamości użytkownika była marginalna. Z czasem standardem stało się podejście oparte na sesjach, gdzie serwer tworzył w pamięci operacyjnej obiekt sesji, a do przeglądarki klienta wysyłał jedynie identyfikator zapisywany w pliku cookie. Choć bezpieczne, rozwiązanie to utrudniało skalowanie poziome w nowoczesnych architekturach mikroserwisowych [8].

Współczesne podejście do cyberbezpieczeństwa coraz częściej opiera się na paradygmacie *Zero Trust* (architekturze zerowego zaufania), szczegółowo zdefiniowanym w publikacjach instytutu NIST [3]. Koncepcja ta zrywa z tradycyjnym modelem "zaufanej sieci wewnętrznej". W modelu *Zero Trust* zakłada się, że zagrożenie może pojawić się zewsząd, dlatego każde, nawet wewnętrzne żądanie dostępu do zasobów musi zostać rygorystycznie zweryfikowane. Oznacza to, że uwierzytelnienie (weryfikacja tożsamości) przy wejściu do systemu nie jest wystarczające – niezbędna jest ciągła weryfikacja uprawnień (autoryzacja) przy każdym pojedynczym zapytaniu [4]. Aby realizować ten postulat wydajnie, branża oprogramowania odeszła od obciążających serwery sesji na rzecz lekkich, kryptograficznie podpisanych poświadczeń przesyłanych w nagłówkach protokołu HTTP [15].

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie oraz implementacja autorskiego, wysoce zoptymalizowanego systemu autoryzacji w środowisku .NET, opartego o funkcjonalność tokenów. Stworzone rozwiązanie stanowi alternatywę dla powszechnego standardu JWT [2], optymalizując narzut sieciowy oraz proces walidacji ról za pomocą operacji bitowych.

Praca została podzielona na logiczne etapy. Wymagane fundamenty teoretyczne oraz krytykę obecnych standardów umieszczono w Rozdziale 2. Zaprojektowana abstrakcyjna architektura, sformalizowane wzory matematyczne operacji kryptograficznych i diagramy przepływów opisane zostały w Rozdziale 3. Następnie w Rozdziale 4 przeanalizowano konkretną implementację programistyczną w języku C#, powołując się na zasady czystego kodu [9]. Rozdział 5 dowodzi uniwersalności rozwiązania, proponując konkretne ścieżki wdrożenia w istniejących projektach oraz analizę wydajnościową popartą wykresami. Dopełnieniem

procesu inżynierskiego jest testowanie rozwiązania i analiza jego bezpieczeństwa w kontekście standardów branżowych, takich jak OWASP Top 10 [1], ujęta we wnioskach końcowych w Rozdziale 6.

2 Przegląd technologii i stan wiedzy

Aby w pełni zrozumieć architekturę proponowanego rozwiązania, konieczne jest omówienie fundamentów teoretycznych oraz technologii wykorzystywanych w nowoczesnych aplikacjach webowych. Rozdział ten wprowadza pojęcia protokołu HTTP, dogłębnie analizuje ewolucję mechanizmów utrzymania stanu, poddaje krytyce popularny standard JWT oraz definiuje stos technologiczny wybrany do implementacji autorskiego systemu, przygotowując tym samym grunt pod zaawansowaną analizę architektoniczną w kolejnym rozdziale.

2.1 Uwierzytelnianie a autoryzacja

W inżynierii bezpieczeństwa oprogramowania pojęcia uwierzytelniania i autoryzacji są często używane zamiennie, co stanowi poważny błąd terminologiczny. Zrozumienie różnicy między nimi jest kluczowe dla poprawnego modelowania systemów dostępu.

Uwierzytelnianie (AuthN) to proces weryfikacji tożsamości użytkownika lub systemu. Odpowiada na pytanie: "Kim jesteś?". W tradycyjnych systemach proces ten realizowany jest najczęściej poprzez podanie loginu i hasła. W rozwiązaniach bardziej zaawansowanych stosuje się biometrię lub uwierzytelnianie wieloskładnikowe (MFA) [5].

Autoryzacja (AuthZ) to proces decyzyjny następujący zawsze po udanym uwierzytelnieniu. Odpowiada na pytanie: "Czy masz prawo to zrobić?". Nawet jeśli tożsamość użytkownika została poprawnie zweryfikowana, system autoryzacyjny musi sprawdzić, czy jego rola uprawnia go do dostępu do konkretnego zasobu (np. usunięcia rekordu z bazy danych) [13]. Niniejsza praca skupia się na optymalizacji obu tych procesów, ze szczególnym naciskiem na minimalizację kosztów obliczeniowych podczas ciągłej autoryzacji zapytań sieciowych.

2.2 Protokół HTTP i problem braku stanu

Protokół HTTP, będący fundamentem komunikacji w sieci Web, z definicji jest protokołem bezstanowym (ang. *Stateless*). Oznacza to, że każde żądanie wysyłane przez klienta do serwera traktowane jest jako całkowicie niezależna operacja [15]. Serwer na poziomie sieciowym nie przechowuje żadnych informacji o poprzednich interakcjach z danym klientem.

Z inżynierskiego punktu widzenia bezstanowość HTTP jest jego ogromną zaletą. Pozwala ona na szybkie obsługiwanie tysięcy jednoczesnych połączeń bez drastycznego przyrostu zużycia pamięci operacyjnej (RAM) serwera. Wymusza to jednak na twórcach oprogramowania aplikacyjnego budowę mechanizmów, które do każdego żądania dołączą dowód tożsamości użytkownika w celu płynnej nawigacji.

2.3 Ewolucja: Od sesji stanowych do bezstanowych tokenów

Początkowo standardem utrzymywania ciągłości pracy użytkownika było podejście oparte na sesjach (ang. *Session-based Authentication*). W tym modelu serwer, po zweryfikowaniu hasła, alokował w swojej pamięci operacyjnej obiekt sesji, a klientowi odsyłał jedynie krótki identyfikator (tzw. *Session ID*) za pomocą mechanizmu ciasteczek (ang. *Cookies*). Przeglądarka automatycznie dołączała ten identyfikator do kolejnych zapytań, a serwer za każdym razem musiał przeszukiwać swoją pamięć lub bazę danych, aby upewnić się, że sesja jest nadal ważna.

Złotą erę tego podejścia zakończył rozwój architektury rozproszonej i chmury obliczeniowej. W środowiskach mikroserwisowych, gdzie ruch rozkładany jest na wiele niezależnych instancji serwerowych (skalowanie poziome), stanowe sesje stały się tzw. wąskim gardłem. Jeśli pierwsze żądanie użytkownika trafiło na serwer A, który zapisał sesję, a kolejne na serwer B, uwierzytelnienie kończyło się błędem.

Rozwiązaniem tego problemu stały się tokeny. W modelu *Token-based Authentication* serwer podpisuje pakiet informacji reprezentujący tożsamość użytkownika, a następnie odsyła go do klienta. Klient przechowuje token i dołącza go do kolejnych żądań HTTP (najczęściej w nagłówku *Authorization*). Różnice w przepływie sterowania obu mechanizmów zobrazowano szczegółowo na Rysunku 2.1.

Jak widać na powołanym schemacie, podejście oparte na tokenach całkowicie odciąża bazę danych z procesu ciągłej autoryzacji. Zamiast odpytywać zasoby dyskowe o stan sesji, serwer samodzielnie weryfikuje podpis matematyczny dostarczony wraz z żądaniem HTTP, co drastycznie obniża czas odpowiedzi.

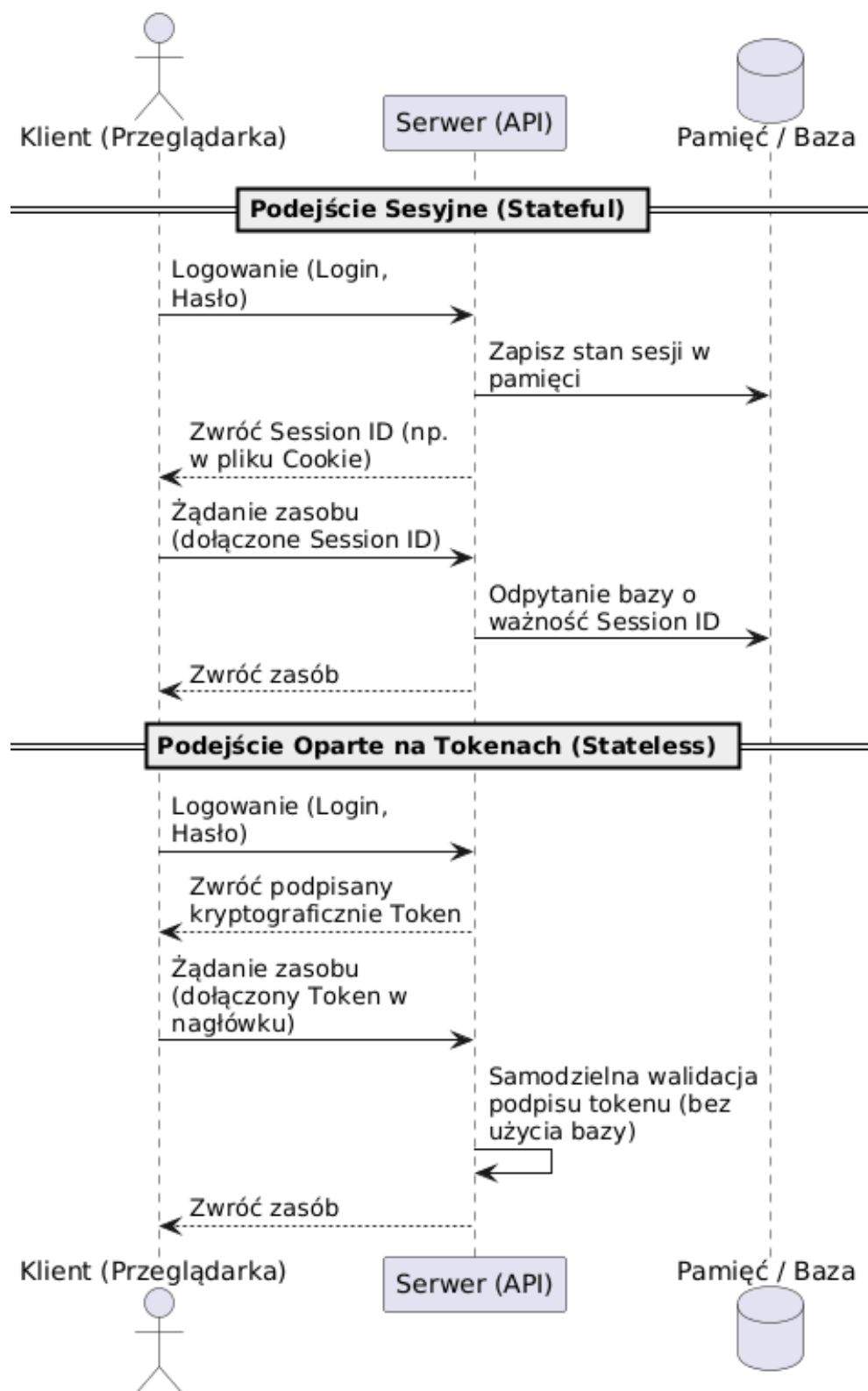
2.4 Anatomia standardu JSON Web Token (JWT)

Najpopularniejszym rynkowym formatem tokenu jest obecnie standard JWT, zdefiniowany oficjalnie w dokumencie RFC 7519 [2]. Klasyczny token JWT to długi ciąg znaków (często przekraczający 500 bajtów), który składa się z trzech części oddzielonych kropkami:

1. Nagłówek (Header): Definiuje typ tokenu oraz algorytm kryptograficzny użyty do podpisu. Zazwyczaj ma postać prostego obiektu JSON:

```
1 {  
2   "alg": "HS256",  
3   "typ": "JWT"  
4 }
```

2. Ładunek (Payload): Miejsce przechowywania oświadczeń (ang. *claims*), czyli faktycznych danych o użytkowniku, takich jak jego identyfikator (*sub*), data wygaśnięcia (*exp*)



Rysunek 2.1. Porównanie architektury stanowej (sesyjnej) z bezstanową (tokenową)

czy posiadane uprawnienia. W standardzie JWT role przesyłane są zazwyczaj jako pełne ciągi tekstowe:

```
1 {  
2   "sub": "1234567890",  
3   "name": "Jan_Kowalski",  
4   "roles": ["admin", "moderator", "user"],  
5   "exp": 1516239022  
6 }
```

3. Podpis (Signature): Kryptograficzny skrót obliczony z zakodowanego nagłówka i ładunku przy użyciu klucza tajnego [7].

Krytyczna analiza tego standardu uwidacznia jego redundancję w zamkniętych środowiskach mikroserwisowych. Przesyłanie w każdym pojedynczym żądaniu nagłówka informującego o typie algorytmu jest zbędnym narzutem danych, ponieważ aplikacja odczytująca token posiada już tę wiedzę z własnej konfiguracji wewnętrznej. Ponadto, tekstowy zapis ról potęguje objętość ładunku. Wyeliminowanie tych nadmiarowych elementów stało się główną motywacją do stworzenia autorskiego formatu opisanego w kolejnych rozdziałach niniejszej pracy.

2.5 Paradygmat Inversion of Control i mechanizm DI

Zastosowany w pracy stos technologiczny wymusza na programistach przestrzeganie dobrych praktyk projektowych [17]. Jednym z najważniejszych paradygmatów zastosowanych w architekturze biblioteki jest odwrócenie sterowania (IoC) realizowane poprzez wzorzec wstrzykiwania zależności (DI) [14].

Zasada ta jest bezpośrednio powiązana z literą "D" w akronimie SOLID [9], która mówi, że moduły wysokopoziomowe nie powinny zależeć od modułów niskopoziomowych, a obie warstwy powinny opierać się na abstrakcjach. Zamiast bezpośrednio tworzyć instancje obiektów (np. połączeń do bazy danych) wewnątrz klas przy użyciu słowa kluczowego **new**, komponenty systemu deklarują swoje zależności poprzez interfejsy w konstruktorach.

W ekosystemie ASP.NET Core wbudowany kontener DI samodzielnie zarządza cyklem życia obiektów [4]. Rozwiązanie to jest krytyczne dla autorskiego systemu autoryzacji, ponieważ zapewnia mu wymaganą hermetyzację, wysoką testowalność oraz modułowość pozwalającą na integrację z dowolnym środowiskiem sieciowym.

2.6 Środowisko uruchomieniowe i persystencja danych

Do realizacji projektu wybrano środowisko uruchomieniowe **.NET** oraz język **C#**. Platforma ta, rozwijana przez firmę Microsoft, stanowi obecnie jeden z najbardziej zaawansowanych

i wydajnych ekosystemów do budowy wielowątkowych aplikacji serwerowych klasy *Enterprise*.

Istotnym elementem architektury jest warstwa persystencji danych. Interakcja z relacyjną bazą danych odbywa się za pośrednictwem technologii Entity Framework Core (EF Core) [6]. Jest to zaawansowane narzędzie klasy ORM, które rozwiązuje problem tzw. niedopasowania impedancji (ang. *impedance mismatch*) pomiędzy światem programowania obiektowego a tabelami bazy danych.

Wykorzystanie EF Core pozwala na operowanie na bazie bez konieczności pisania jawnych zapytań w języku SQL. Kodowanie odbywa się przy pomocy wbudowanej składni LINQ, a framework automatycznie tłumaczy te komendy na bezpieczny i zoptymalizowany pod dany silnik dialekt SQL. Użycie tego narzędzia automatycznie zabezpiecza autorski system autoryzacji przed jednym z najgroźniejszych ataków sieciowych – *SQL Injection* – ponieważ wszystkie wartości wstawiane do bazy są domyślnie traktowane przez warstwę ORM jako bezpieczne, parametryzowane dane wejściowe.

3 Projekt i architektura systemu

Architektura omawianego systemu została zoptymalizowana pod kątem redukcji narzutu sieciowego oraz błyskawicznej walidacji zapytań. W tym rozdziale zaprezentowano projekt mechanizmów wewnętrznych, sformalizowane modele matematyczne algorytmów oraz przepływy procesów decyzyjnych. Zgodnie z dobrymi praktykami dokumentacji architektonicznej [10], celowo pominięto na tym etapie bezpośrednie implementacje kodowe, skupiając się na czystej logice systemowej.

3.1 Koncepcja autorskiego tokenu

W przeciwieństwie do powszechnych standardów rynkowych, autorski token został zaprojektowany z myślą o całkowitej minimalizacji przesyłanych bajtów. Jest to zoptymalizowany ciąg znaków składający się zaledwie z dwóch segmentów: ładunku (B) zawierającego właściwe oświadczenia i dane użytkownika oraz sumy kontrolnej (S).

Proces powstawania tokenu T sformalizowano za pomocą następującego równania strukturalnego:

$$T = \text{Base64Url}(B) + "." + \text{Base64Url}(S) \quad (3.1)$$

Zgodnie ze wzorem (3.1), zmienne oznaczają odpowiednio:

- T – finalny ciąg znaków reprezentujący token autoryzacyjny, przesyłany w nagłówku zapytania HTTP.
- B – ładunek (ang. *Body/Payload*), będący tekstową reprezentacją zserializowanego obiektu JSON, przechowującego dane sesji (np. zdekodowaną maskę ról oraz czas wygaśnięcia).
- S – suma kontrolna (ang. *Signature*), będąca binarnym wynikiem operacji kryptograficznej zabezpieczającej ładunek przed modyfikacją.
- $\text{Base64Url}()$ – funkcja kodująca binarne dane do bezpiecznego formatu tekstowego.

Kluczowym zabiegiem inżynierskim zastosowanym w tym algorytmie jest użycie kodowania **Base64Url** zamiast standardowego formatu Base64. Standardowe kodowanie wprowadza znaki takie jak $+$ oraz $/$, które posiadają specjalne, zastrzeżone znaczenie w adresach URL i nagłówkach HTTP (co mogłoby prowadzić do błędów parsowania na poziomie sieci). Algorytm Base64Url rozwiązuje ten problem, zastępując je odpowiednio znakami $-$ (myślnik) oraz $_$ (podkreślenie), a także usuwa znaki dopełnienia $=$, co gwarantuje pełną, bezbłędną kompatybilność z protokołem sieciowym.

3.2 Algorytm podpisu cyfrowego

Bezpieczeństwo nowoczesnego systemu dostępu opiera się na bezwzględny zapewnieniu integralności i niezaprzeczalności emitowanych poświadczeń. Do generowania sumy kontrolnej (S) w zaprojektowanym rozwiązaniu wykorzystano bezpieczny algorytm symetryczny HMAC w oparciu o silną funkcję skrótu SHA-256 [11].

Wybór kryptografii symetrycznej podyktowany jest niespotykaną wydajnością oraz specyfiką aplikacji monolitycznych, gdzie serwer wydający i weryfikujący to ten sam podmiot, posiadający w 100% bezpieczny dostęp do własnej pamięci operacyjnej. Pełny proces generowania cyfrowego podpisu σ wyrażono wzorem matematycznym:

$$\sigma = \text{Base64Url}(\text{HMAC}_{\text{SHA256}}(K, P)) \quad (3.2)$$

Jak wynika ze wzoru (3.2), parametry funkcji mieszającej przyjmują następujące wartości:

- σ – ostateczny, tekstowy podpis dołączany po kropce na końcu tokenu.
- K – klucz tajny (ang. *Secret Key*), będący ciągiem znaków o wysokiej entropii, zdefiniowanym z zewnątrz w bezpiecznej konfiguracji środowiskowej serwera. Klucz ten nigdy, pod żadnym pozorem nie jest przesyłany do klienta.
- P – ładunek wejściowy poddawany szyfrowaniu, czyli zakodowana już wcześniej w `Base64Url` część danych tekstowych ($P = \text{Base64Url}(B)$).
- $\text{HMAC}_{\text{SHA256}}$ – kryptograficzna funkcja mieszająca, która w bezpieczny sposób łączy klucz tajny z wiadomością, będąc algorytmem odpornym na popularne wektory ataków, w tym ataki typu *Length Extension Attack* [7].

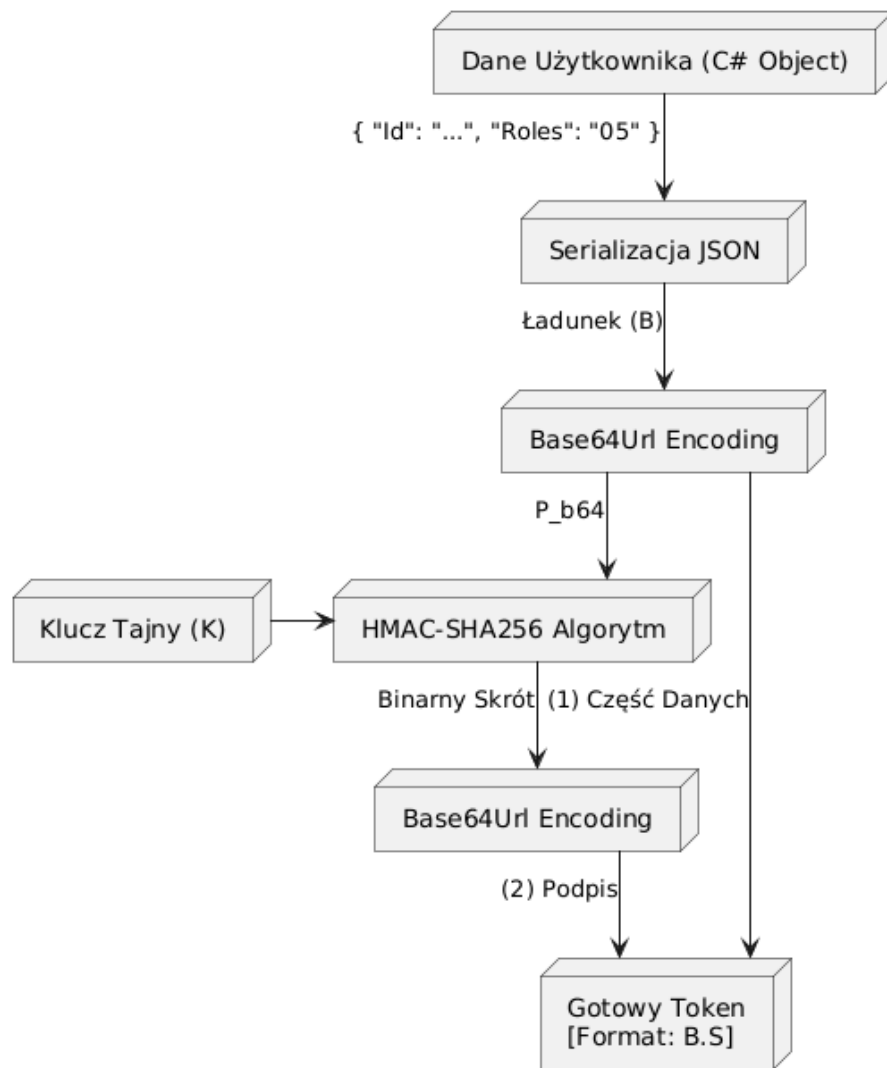
Przepływ danych w procesie generowania tokenu, obrazujący ścieżkę od momentu pobrania natywnego obiektu `C#` po uzyskanie końcowego, złączonego ciągu znaków do wysyłki, zaprezentowano w formie graficznej na Rysunku 3.1.

Opierając się na schemacie, wyraźnie widać dwuetapowość procesu. Po serializacji obiektu JSON ładunek rozgałęzia się – z jednej strony trafia jako pierwsza część docelowego poświadczenia, a z drugiej strony stanowi materiał wejściowy (tzw. sól algorytmiczną) dla silnika HMAC, generującego unikalny dla tego ładunku podpis.

3.3 Weryfikacja tokenu i integralność

Zrozumienie procesu weryfikacji po stronie serwera jest kluczowe z perspektywy analizy bezpieczeństwa opisywanej architektury. Weryfikacja bezstanowa nie polega na kryptograficznym "odszyfrowywaniu" dostarczonej sumy kontrolnej, lecz na deterministycznym procesie jej matematycznej reprodukcji.

Kiedy serwer chroniony odbiera token z żądania HTTP, rozdziela go programowo używając znaku kropki jako dedykowanego separatora. Następnie, korzystając z przechwyconego



Rysunek 3.1. Architektura kryptograficzna generowania tokenu

z sieci ładunku P oraz zachowanego w swojej pamięci własnego klucza symetrycznego K , algorytm bezpieczeństwa ponownie wykonuje opisaną wcześniej funkcję $\text{HMAC}_{\text{SHA256}}$. Jeśli nowo wyliczona w pamięci RAM sygnatura σ' jest znakowo i matematycznie tożsama z sygnaturą σ nadesłaną przez klienta (czyli spełniony jest warunek $\sigma' == \sigma$), serwer nabiera 100% pewności, że dane nie uległy zewnętrznej manipulacji. Ewentualna, sfabrykowana zmiana choćby jednego bitu w tekście ładunku (np. próba nielegalnej zmiany identyfikatora przypisanej roli) spowoduje drastyczną zmianę ostatecznie wyliczonego skrótu wskutek tzw. zjawiska lawinowego (ang. *Avalanche Effect*). Poskutkuje to niezgodnością podpisów i natychmiastowym odrzuceniem szkodliwego żądania HTTP z zastrzeżonym kodem autoryzacji 401 Unauthorized.

3.4 Zarządzanie rolami przy użyciu masek bitowych

Największą sprzętową innowacją zaprojektowanego systemu autoryzacji jest całkowite odejście od tekstowej, listowej reprezentacji uprawnień użytkownika. Nowatorski model zarządzania dostępem opiera się tu na matematycznej teorii zbiorów oraz systemie wag dwójkowych [12].

Niech R stanowi zbiór wszystkich dostępnych w całym systemie ról, a każda unikalna rola $r \in R$ posiada nadany własny, unikalny indeks liczbowy $ID_r \in \{0, 1, 2, \dots, n\}$. Zbiór ról przypisanych w bazie danych do konkretnego użytkownika oznaczmy jako podzbiór $U \subseteq R$. Unikalna maska bitowa M reprezentująca pełen wachlarz praw dla danego użytkownika obliczana jest algorytmicznie jako suma wag potęg liczby 2:

$$M = \sum_{r \in U} 2^{ID_r} \quad (3.3)$$

Aby zilustrować działanie wzoru (3.3) w praktyce, można przyjąć prosty system posiadający predefiniowane role "user" ($ID = 0$) oraz "admin" ($ID = 2$). Jeżeli rozpatrywany użytkownik posiada przyznane jednocześnie obie z nich, jego wyliczona maska sumaryczna wynosi:

$$M = 2^0 + 2^2 = 1 + 4 = 5 \quad (3.4)$$

Liczba dziesiętna 5 jest następnie w tle konwertowana przez kompilator do formatu szesnastkowego (reprezentacja 05) i w takiej właśnie, wysoce skompresowanej formie tekstowej umieszczana wewnątrz ładunku tokenu JSON.

Kiedy weryfikowany użytkownik żąda dostępu do konkretnego zasobu API, zabezpieczonego określoną przez programistę rolą r_{req} , chroniący ten zasób serwer weryfikujący nie musi parsować długich łańcuchów znaków ani przeszukiwać list. Wykonuje on wyłącznie jedną, natywną dla każdego procesora komputerowego operację iloczynu bitowego (AND). Dostęp sieciowy zostaje udzielony wtedy i tylko wtedy, gdy przez procesor weryfikujący spełniony

jest warunek:

$$(M \text{ AND } 2^{ID_{req}}) \neq 0 \quad (3.5)$$

Zastosowanie operatora weryfikacji ze wzoru (3.5) całkowicie omija kosztowną alokację i zwalnianie pamięci serwera, czyniąc proces weryfikacji uprawnień niezwykle, wręcz sprzętowo wydajnym.

3.5 Analiza złożoności obliczeniowej i pamięciowej

Sformalizowanie struktury ról do minimalistycznej postaci masek bitowych przynosi inżyniersko wymierne korzyści, co można z łatwością udowodnić za pomocą standardowej w informatyce notacji asymptotycznej (*Big O Notation*).

W standardowym podejściu stosowanym przez rynkowe rozwiązania (np. klasyczny framework JWT w ASP.NET), role posiadane przez logującego się użytkownika weryfikowane są poprzez wielokrotne, iteracyjne przeszukiwanie tablicy ciągów znaków (np. za pomocą metody *Contains*). Dla N ról zdefiniowanych dla użytkownika oraz M ról wymaganych przez docelowy punkt końcowy, złożoność obliczeniowa pesymistycznego (najgorszego) przypadku walidacji wynosi aż $O(N \cdot M)$. Ponadto, operacje wykonywane na łańcuchach znaków (ang. *string comparisons*) dodatkowo, z każdym requestem, obciążają pamięć sterty (ang. *Heap*) oraz aktywują wirtualną maszynę odśmiecającą pamięć (tzw. *Garbage Collector*).

Z kolei w opracowanym mechanizmie autorskim, po początkowym zdekodowaniu krótkiej maski szesnastkowej, proces walidacji uprawnień sprowadza się fizycznie do pojedynczego wywołania operatorów sprzętowych ALU wewnątrz procesora. Złożoność obliczeniowa tak prymitywnego porównania bitowego, bez względu na rosnącą wielkość słownika ról w dużym systemie, jest bezwzględnie stała i wynosi zawsze $O(1)$. Wartość ta stanowi teoretyczne i praktyczne minimum dla systemów dostępowych, a sama złożoność pamięciowa została na tym etapie zredukowana do czasowej alokacji zaledwie pojedynczych zmiennych typów prostych (ang. *Value Types*) bezpośrednio na szybkim stosie aplikacji (ang. *Stack*).

3.6 Cykl życia sesji i rotacja tokenów odświeżających

Utrzymywanie niezwykle krótkiego czasu życia tzw. tokenu dostępowego (ang. *Access Token* - zazwyczaj w przedziale od kilku do kilkunastu minut) stanowi absolutny fundament polityki bezpieczeństwa w każdym modelu bezstanowym. Restrykcyjne ograniczenie czasu życia wprost i drastycznie minimalizuje podatne okno czasowe, w którym ewentualnie przechwycony przez włamywacza z sieci lokalnej token mógłby posłużyć do wyrządzenia realnych szkód w systemie.

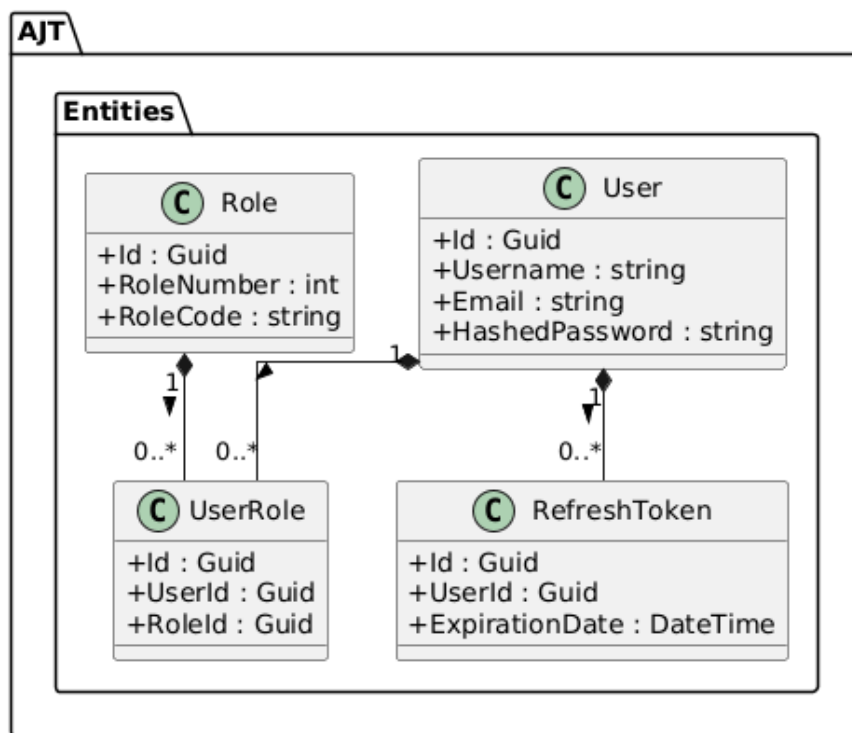
Aby jednak jednocześnie zapewnić wysoką ergonomię i ciągłość pracy bez konieczności nieustannego wymuszania na użytkowniku ponownego wpisywania poświadczeń (hasła), do architektury wprowadzono osobne, długoterminowe tokeny odświeżające (ang. *Refresh Tokens*). Prezentowana architektura zakłada przy tym bezwzględne wdrożenie mechanizmu bezpiecznej rotacji (ang. *Refresh Token Rotation*) [13].

W praktyce proces ten przebiega następująco: gdy właściwy token dostępowy w przeglądarce traci swoją ważność, aplikacja kliencka dyskretnie wysyła na dedykowany punkt końcowy serwera zachowany w pamięci token odświeżający. Ze względów bezpieczeństwa jest to jedyny etap autoryzacji, w którym zaprojektowany system wykonuje świadomą operację stanową (odpytuje dyskową relacyjną bazę danych). Po rutynowym zweryfikowaniu integralności kryptograficznej łańcucha znaków, serwer upewnia się, że wskazany token odświeżający fizycznie nadal istnieje w jego bazie i nie uległ z kolei własnemu przedawnieniu. Jeżeli pełna weryfikacja powiedzie się, stary token jest natychmiast i bezzwłocznie usuwany z systemu, a serwer w zamian emituje dla przeglądarki całkowicie nową parę zaktualizowanych kluczy (nowy, krótki Access Token oraz wygenerowany na nowo Refresh Token). Przeprowadzenie tego transakcyjnego mechanizmu matematycznie gwarantuje, że skradziony stary token odświeżający będzie dla atakującego bezużyteczny lub jednorazowy – w momencie, gdy tylko prawowity użytkownik lub system wygeneruje w tle nową sesję rotacyjną, skradziony stary identyfikator zostanie bezpowrotnie unieważniony, zamykając sesję.

3.7 Diagram klas: Warstwa danych

Z uwagi na zastosowanie przy inżynierii kodu wyższych wzorców projektowych, cały zbudowany system podzielono na odseparowane, hermetyczne pakiety logiczne, co szczegółowo zilustrowano za pomocą standardowej graficznej notacji UML [8]. Zaprezentowany Rysunek 3.2 prezentuje encje bazodanowe (klasy z adnotacjami języka C#) tworzące model struktury relacyjnej bazy.

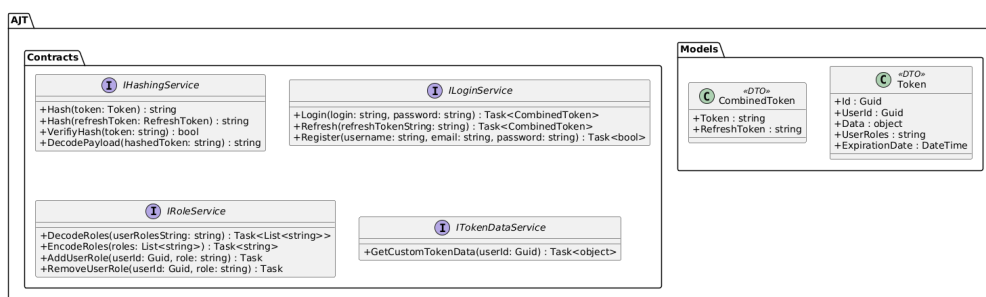
Jak widać na diagramie warstwy danych, centralnym punktem całego modułu autoryzacyjnego jest encja **User**, przechowująca wrażliwe poświadczenia logowania takie jak nazwa i zabezpieczone hasło (**HashedPassword**). Jest ona silnie, relacyjnie powiązana z klasą **RefreshToken** (stanowiąc klasyczną relację jeden-do-wielu, co pozwala jednemu użytkownikowi na bycie aktywnym w wielu środowiskach naraz). Ponadto, wdrożono znormalizowane przypisanie zdefiniowanych z poziomu konfiguracji uprawnień poprzez standardową dla baz SQL tabelę asocjacyjną **UserRole**, stanowiącą jedyny, bezpieczny most pomiędzy kontem a matematycznie obsługiwaną tabelą bazową słownika **Role**. Taka granulacja tabel gwarantuje najwyższą higienę danych w szablonie architektury monolitycznej.



Rysunek 3.2. Diagram klas warstwy danych (Entities)

3.8 Diagram klas: Kontrakty i modele

Kolejny schemat strukturalny, widoczny na Rysunku 3.3, obrazuje projekt bezstanowych obiektów transferu danych (tzw. DTO), służących do transportu i operowania wartościami poza ścisłym uściskiem repozytoriów bazodanowych.

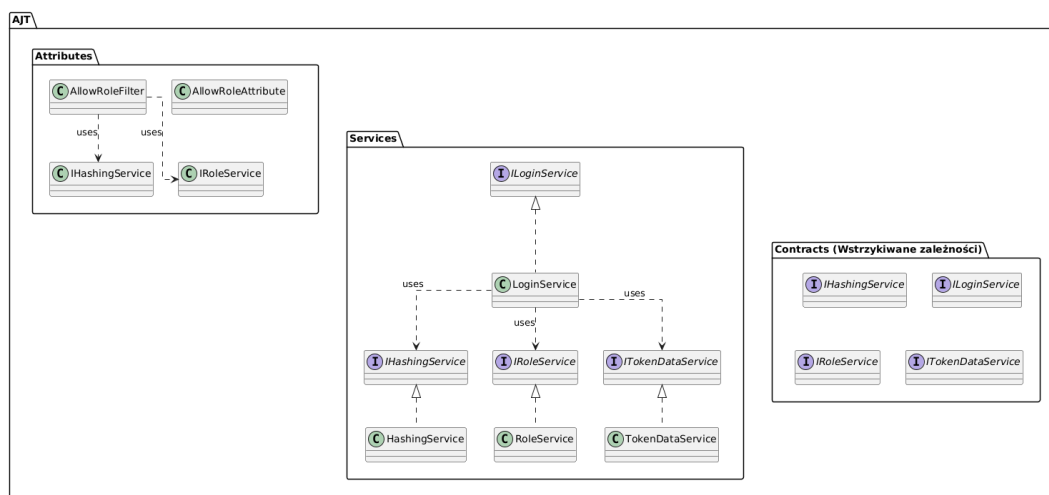


Rysunek 3.3. Diagram klas warstwy kontraktów i modeli

Model `CombinedToken` pełni kluczową rolę jako obiekt zbiorczy wysyłany w odpowiedzi na poprawne logowanie, agregując w sobie właściwy obiekt sesji, jak i string odświeżający. Należy tu również dostrzec wyraźną, zgodną z dobrymi praktykami paradygmatu SOLID separację deklaracji abstrakcyjnych interfejsów umów (jak `IHashingService` czy `IRoleService`) od fizycznie implementujących je klas [14]. Wstrzykiwanie w całym systemie wyłącznie powołanych tutaj interfejsów jest rygorystycznym, strukturalnym wymogiem obsługi wbudowanego kontenera DI, co pozwala na ewentualną i szybką podmianę fizycznych serwisów bez konieczności refaktoryzacji kodu wyższych warstw sterujących.

3.9 Diagram klas: Logika i punkty dostępu

Ostatni z serii zaprojektowanych schematów statycznych (Rysunek 3.4) skupia się na zobrażowaniu docelowych relacji operacyjnych zgrupowanych w wymagającej warstwie biznesowej całego rozwiązania.



Rysunek 3.4. Diagram klas implementacji logiki biznesowej

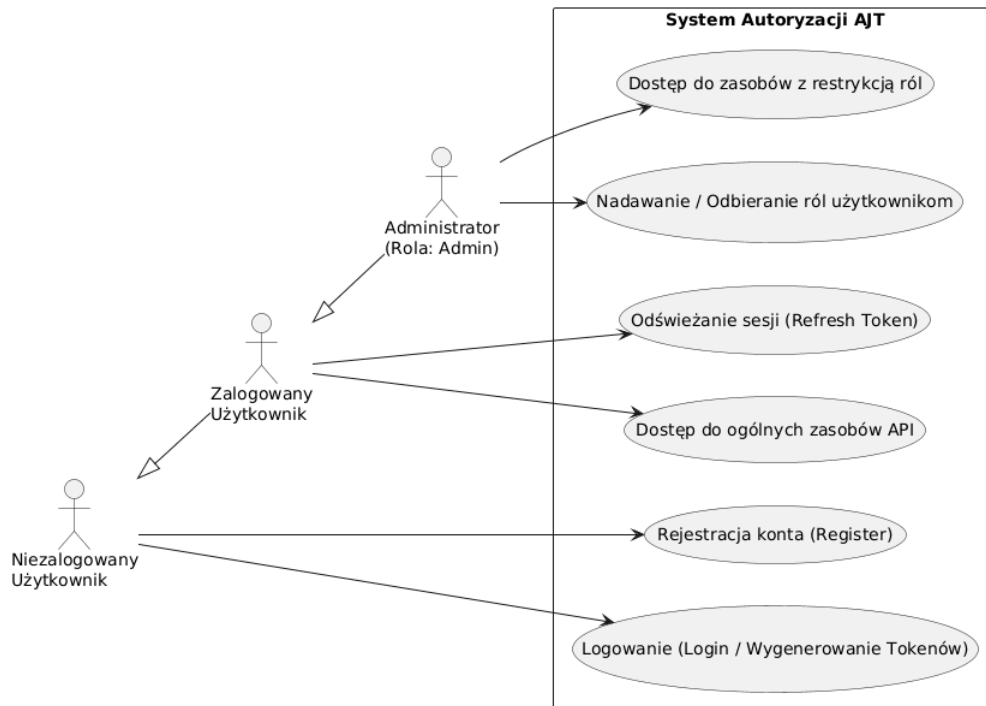
Diagram ten precyzyjnie obrazuje złożoną zależność kompozycyjną między autorskimi, stworzonymi atrybutami (np. `AllowRoleAttribute`) i wbudowanymi, polimorficznymi filtrami autoryzacyjnymi przechwytyjącymi natywne żądania środowiska ASP.NET. Widać tu wyraźnie orkiestrującą rolę wydzielonej klasy kontrolera `LoginService`, która wewnątrz swoich asynchronicznych metod deleguje ciężką pracę weryfikacyjną na wstrzyknięte do niej i obsługiwane niezależnie instance konkretnych implementacji interfejsów szyfrujących (takich jak np. widoczny `HashingService`). Tak rozplanowane w strukturze klas granice wprost zapewniają spełnienie inżynierskiego postulatu jednej odpowiedzialności (`Single Responsibility Principle`).

3.10 Diagram przypadków użycia

Podczas analizy domeny zidentyfikowano i wyodrębniono ostatecznie trzech głównych aktorów fizycznych wchodzących w celową interakcję ze stworzonym modułem sieciowym (są to kolejno: gość niezalogowany, użytkownik zalogowany do systemu, oraz administrator o podwyższonych uprawnieniach).

Jak wprost zobrazowano na czytelnym Rysunku 3.5, hierarchiczna architektura autoryzacji opiera się w dużej mierze na tzw. kaskadowym dziedziczeniu weryfikowanych uprawnień. Wyższy, uprzywilejowany poziom dostępu (administrator) naturalnie wchłania i dziedziczy wszystkie podstawowe kompetencje poziomu niższego (może podtrzymać własną sesję tak jak zwykły pracownik), jednak to posiadana przezeń odpowiednia, dodatkowa maska bitowa

jako jedyna odblokowuje administracyjne, czułe punkty dostępowe API chronione dedykowanymi filtrami blokującymi.

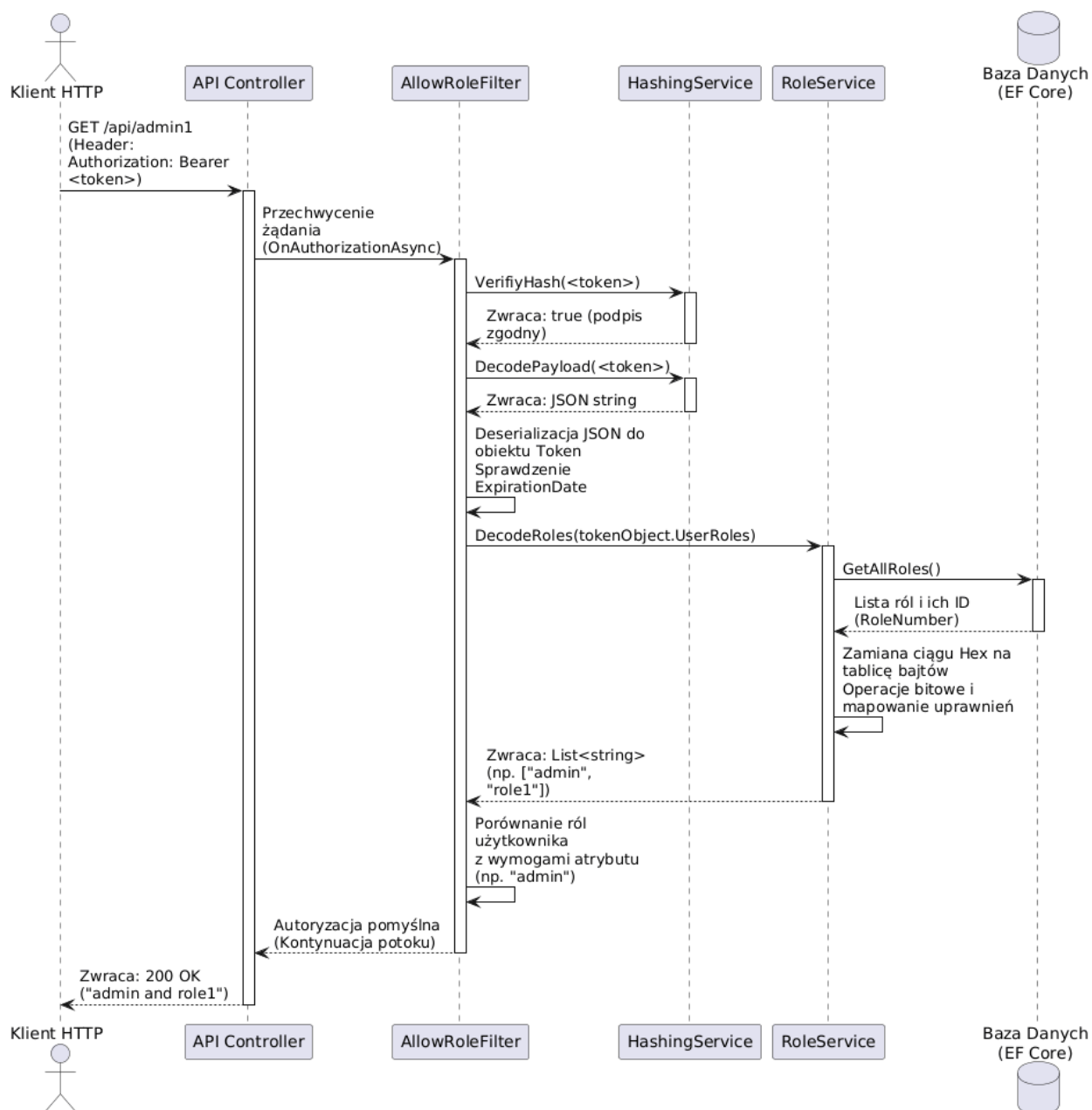


Rysunek 3.5. Diagram przypadków użycia systemu

3.11 Diagram sekwencji weryfikacji uprawnień

Czasowy, w pełni sekwencyjny i uporządkowany przepływ natychmiastowej weryfikacji żądanego dostępu do chronionych zasobów przedstawiono krok po kroku na wygenerowanym Rysunku 3.6.

Proces ten w całości aktywuje się asynchronicznie dokładnie w momencie fizycznego dostarczenia obcego żądania HTTP do potoku serwera, tuż przed rutynowym wywołaniem przez framework właściwej dla danego kontrolera logiki domenowej. Schemat szczegółowo przedstawia pełen, nierozzerwany cykl życia autorskiej walidacji: od surowego odebrania nagłówków, poprzez natychmiastową weryfikację skrótu kryptograficznego HMAC za pomocą dostarczonej z zewnątrz `AJTOptions.Secret`, przez szybkie odzyskanie obiektu, binarne zdekodowanie matematycznej maski tekstowych ról wewnątrz serwisu, aż po ostateczne przerwanie odpowiedzi błędem (Forbidden) lub zgodną kontynuację wywołania potoku wykonawczego.



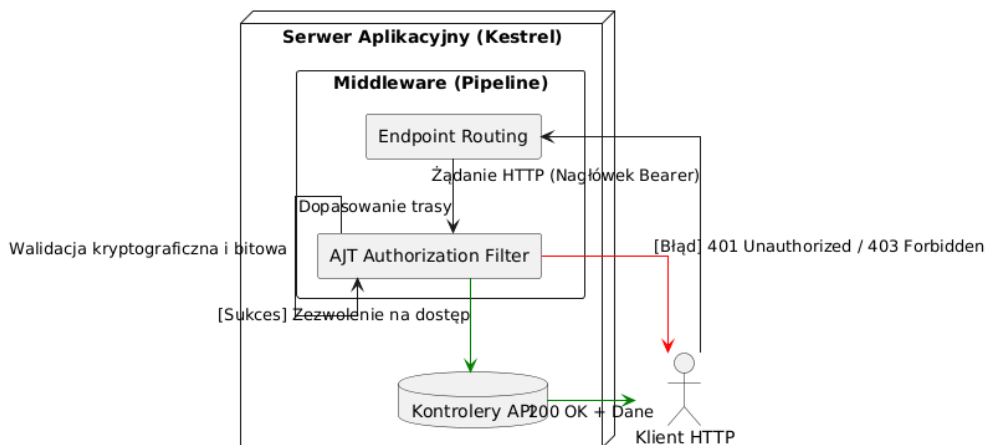
Rysunek 3.6. Diagram sekwencji weryfikacji uprawnień

4 Implementacja rozwiązania

Mając na uwadze przygotowany i opisany w poprzednim rozdziale kompletny, teoretyczny model architektury klas, przystąpiono z kolei do właściwego, fizycznego programowania w obiekto-orientowanym środowisku języka C#. Zgodnie ze ścisłymi wytycznymi czystego kodu [9] oraz nowoczesnej zasady inżynierskiego rzemiosła oprogramowania, w tej, zasadniczej części pracy zaprezentowano wyłącznie wyselekcjonowane, kluczowe dla logiki działania fragmenty wyprodukowanego kodu źródłowego. Zrezygnowano w niej z przytaczania całościowych i niezbędnych deklaracji klas czy pakietów na rzecz wysoce wnikliwej, akademickiej analizy zaimplementowanych algorytmów autoryzacyjnych.

4.1 Logika biznesowa i potok ASP.NET Core

Zanim jakikolwiek zalogowany użytkownik skutecznie uzyska sieciowy dostęp do docelowych, chronionych procedur i zasobów dyskowych, serwująca aplikacja musi najpierw bezpiecznie przechwycić i przetworzyć jego surowe zapytanie. Rozbudowana architektura serwerowej platformy ASP.NET Core od samego zarania w sposób natywny opiera się na wydajnym potoku asynchronicznego przetwarzania (ang. *Request Pipeline*), zbudowanym wielowarstwowo z różnego rodzaju konfigurowalnego oprogramowania pośredniczącego (tzw. *Middleware*) oraz rejestrowanych dynamicznie w kodzie filtrów. Zrealizowany system biblioteki płynnie i w całości wstrzykuje całą swoją opisaną logikę decyzyjną precyzyjnie w ten ustandaryzowany potok, zajmując krytyczne miejsce na grafie tuż przed zaplanowanym przekazaniem sterowania żądaniem do kontrolera biznesowego natywnej aplikacji webowej, co czytelnie zilustrowano na przygotowanym Rysunku 4.1.



Rysunek 4.1. Integracja filtra autoryzacyjnego z potokiem żądań ASP.NET Core

Zanim jednak osadzony filtr blokujący będzie w stanie technicznie zwalidować surowe żądanie wejściowe HTTP (jak widać to po lewej stronie zamieszczonego schematu integracji), nowy użytkownik musi zawsze najpierw skutecznie wygenerować swoje prywatne, cyfrowe poświadczenie sieciowej autentyczności, komunikując się w tym celu wpierw z wydzielonym, ogólnodostępnym dla wszystkich klientów kontrolerem dedykowanego serwisu logowania bazy danych.

4.2 Rejestracja i bezpieczne przechowywanie haseł

Niekwestionowanym fundamentem w każdym zamkniętym systemie oceny tożsamości jest prawidłowy, pozbawiony luk w dostępie i bezpieczny transakcyjnie proces początkowej rejestracji nowej jednostki (ang. *Provisioning*). Wstrzykiwany serwis programistyczny odpowiedzialny wyłącznie za ten newralgiczny proces, za każdym wywołaniem metody zmuszony jest upewnić się, że dostarczone z zewnątrz formularzowe dane logowania potencjalnego użytkownika pozostają bezsprzecznie unikalne w skali całego systemu, a co ważniejsze z punktu widzenia ochrony prywatności – jego podane w postaci prostej hasło z klawiatury nigdy, pod żadnym pozorem, nie trafia do fizycznego zapisu na dysku bazy danych w niezabezpieczonej, odczytywalnej przez człowieka postaci jawnej tekstowej (tzw. *Plain text*).

W zaprezentowanej na Listingu 4.1 wydzielonej publicznej metodzie sieciowej o nazwie **Register** poprawnie i zgodnie z najlepszymi praktykami zastosowano wbudowany w ekosystem C# natywny interfejs realizujący wielokrotne, kryptograficzne solenie i wielorundowe haszowanie znaków (`_passwordHasher.HashPassword`), co w rzeczywistości operacyjnie gwarantuje spełnienie i wypełnienie rygorystycznych, restrykcyjnych wytycznych polityki bezpieczeństwa dla audytów sieciowych typu OWASP Top 10 [1]. Metoda na wejściu (linie 7-8) wydajnie weryfikuje aktualną zajętość pożądanego loginu z klawiatury oraz wskazanego adresu e-mail, zawsze korzystając zresztą w tym ściśle określonym celu ze skonstruowanego i jednorazowo wydzielonego dla asynchronicznej metody zakresu powołanego wcześniej i stojącego u podstaw aplikacji kontenera odwróconych zależności (DI).

Listing 4.1. Metoda rejestracji użytkownika wraz z szyfrowaniem haseł

```
1 public async Task<bool> Register(string username,
2                                 string email, string password)
3 {
4     using var scope = _scopeFactory.CreateScope();
5     var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
6
7     var existingUserUsername = await userRepo.GetUserByLogin(username);
8     var existingUserEmail = await userRepo.GetUserByLogin(email);
9
10    if (existingUserUsername is not null || existingUserEmail is not null)
11        return false;
12
13    var user = new User
14    {
15        Username = username,
16        Email = email,
17        HashedPassword = _passwordHasher.HashPassword(password)
18    };
19
20    try
21    {
22        await userRepo.AddUser(user);
23        return true;
24    }
25    catch
26    {
27        return false;
28    }
29 }
```

Jak wyraźnie widać wewnątrz samego ciała tej metody, cała ryzykowna z punktu widzenia zerwanych połączeń z bazą danych próba wstawienia ostatecznego rekordu zabezpieczona została szerokim, defensywnym w działaniu blokiem obsługi wyjątków typu `try-catch` (linie 20-27). Zapobiega to całkowicie przed niekontrolowanym, niebezpiecznym "wyciekaniem" tzw. wrażliwych stert śledzenia oprogramowania (ang. *Stack trace*) na stronę klienta API w razie wewnętrzserwerowych awarii relacyjnych kluczy indeksów.

4.3 Proces logowania i weryfikacji tożsamości

Głównym punktem wejścia do chronionej strefy API jest serwis `LoginService`. Odpowiada on za autentykację użytkownika na podstawie dostarczonych poświadczeń logowania.

Na Listingu 4.2 zaprezentowano bazową metodę `Login`. Kluczowym aspektem jej implementacji jest wykorzystanie fabryki zakresów (`IServiceScopeFactory`). Ponieważ autoryzacja może być wywoływana z różnych kontekstów cyklu życia aplikacji, serwis każdorazowo tworzy nowy, odizolowany zakres dla kontenera wstrzykiwania zależności (DI). Dopiero z niego pobierane jest repozytorium użytkowników (`IUserRepo`).

Po pomyślnej weryfikacji skrótu hasła (linia 7), właściwy i złożony obliczeniowo proces kryptograficznego generowania tokenów delegowany jest do prywatnej metody pomocniczej. Taki zabieg architektoniczny znacząco upraszcza implementację interfejsu publicznego i jest zgodny z dobrymi praktykami programowania.

Listing 4.2. Metoda podstawowa autentykacji i weryfikacji hasła

```
1 public async Task<CombinedToken?> Login(string login, string password)
2 {
3     var hashedPassword = _passwordHasher.HashPassword(password);
4     using var scope = _scopeFactory.CreateScope();
5     var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
6     var user = await userRepo.GetUserByLogin(login);
7     if (user is null || user.HashedPassword != hashedPassword)
8         return null;
9     return await CreateCombinedTokenForUser(scope, user);
10 }
```

Właściwy proces budowy tokenu, kompresji ról oraz kryptograficznego podpisywania poświadczeń został zhermetyzowany w prywatnej metodzie pomocniczej `CreateCombinedTokenForUser` (Listing 4.3). Metoda ta przyjmuje jako parametry wygenerowany wcześniej zakres wstrzykiwania zależności (ang. *Scope*) oraz zweryfikowany obiekt użytkownika.

W linii 17 następuje wywołanie serwisu `_roleService.EncodeRoles`, który odpowiada za transformację tekstowych ról w skompresowaną maskę bitową. Następnie system generuje unikalny token odświeżający (*Refresh Token*) i trwale zapisuje go w relacyjnej bazie danych za pomocą narzędzia EF Core. Zwieńczeniem procesu logowania (linie 31-35) jest wywołanie metody `Hash`. Odpowiada ona za serializację struktur obiektowych C# do formatu JSON, ich zakodowanie oraz sprzętowe zabezpieczenie algorytmem HMAC-SHA256, co gwarantuje ochronę przed nieautoryzowaną modyfikacją poświadczeń [12].

Listing 4.3. Budowa, kompresja bitowa i generowanie tokenów kryptograficznych

```
1 private async Task<CombinedToken?> CreateCombinedTokenForUser(  
2     IServiceScope scope, User user)  
3 {  
4     var userRoleRepo = scope.ServiceProvider  
5         .GetRequiredService<IUserRoleRepo>();  
6     var roles = await userRoleRepo.GetUserRoles(user);  
7  
8     var token = new Token  
9     {  
10         Id = Guid.NewGuid(),  
11         UserId = user.Id,  
12         Data = await _tokenDataService.GetCustomTokenData(user.Id),  
13         ExpirationDate = DateTime.Now.Add(_options.Value.  
14             TokenExpirationTime)  
15     };  
16  
17     if (roles is not null)  
18         token.UserRoles = await _roleService.EncodeRoles(  
19             roles.Select(x => x.RoleCode).ToList());  
20  
21     var refreshTokenRepo = scope.ServiceProvider  
22         .GetRequiredService<IRefreshTokenRepo>();  
23  
24     var refreshToken = new RefreshToken  
25     {  
26         UserId = user.Id,  
27         ExpirationDate = DateTime.Now  
28             .Add(_options.Value.RefreshTokenExpirationTime)  
29     };  
30     await refreshTokenRepo.AddRefreshToken(refreshToken);  
31  
32     return new CombinedToken  
33     {  
34         Token = _hashingService.Hash(token),  
35         RefreshToken = _hashingService.Hash(refreshToken)  
36     };  
37 }
```

4.4 Implementacja odświeżania sesji i rotacji tokenów

Zgodnie z wymaganiami bezpieczeństwa, czas życia wydanego tokenu dostępowego (`TokenExpirationTime`) jest celowo krótki. Aby zapewnić ciągłość pracy bez wymuszania na użytkowniku ponownego logowania, zaimplementowano mechanizm rotacji tokenów odświeżających wewnątrz metody `Refresh` (Listing 4.4).

W pierwszej kolejności (linia 3) algorytm weryfikuje poprawność kryptograficzną nadesłanego ciągu. Odrzucenie zmodyfikowanych lub sfałszowanych poświadczeń już na tym etapie pozwala zaoszczędzić zasoby serwera, unikając zbędnych i kosztownych zapytań do bazy danych. Następnie system bezpiecznie deserializuje ładunek JSON do obiektu, wykorzystując blok `try-catch` (linie 7-16) w celu ochrony przed krytycznymi błędami parsowania (HTTP 500).

Kluczowym etapem z punktu widzenia architektury *Zero Trust* jest weryfikacja stanu tokenu w bazie danych (linie 21-23). Jeśli token istnieje i jest nadal ważny, zostaje natychmiast usunięty z repozytorium (linia 25). Ta operacja gwarantuje absolutną jednorazowość tokenu odświeżającego, co skutecznie zamyka wektor ataków typu *Replay Attack* (polegających na przechwyceniu i ponownym użyciu cudzego żądania sieciowego). Proces kończy się wygenerowaniem całkowicie nowej pary kluczy dla zidentyfikowanego użytkownika.

Listing 4.4. Weryfikacja bazy danych i odświeżanie tokenów rotacyjnych

```
1 public async Task<CombinedToken?> Refresh(string refreshTokenString)
2 {
3     if (!_hashingService.VerifyHash(refreshTokenString))
4         return null;
5     // [...]
6     var json = _hashingService.DecodePayload(refreshTokenString);
7     refreshToken = JsonConvert.DeserializeObject<RefreshToken>(json);
8     if (refreshToken is null) return null;
9     // [...]
10    var dbToken = await refreshRepo.GetRefreshToken(refreshToken.Id);
11    if (dbToken is null || dbToken.ExpirationDate < DateTime.Now)
12        return null;
13    await refreshRepo.DeleteRefreshToken(dbToken.Id);
14    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();
15    var user = await userRepo.GetUserById(dbToken.UserId);
16    if (user is null)
17        return null;
18    return await CreateCombinedTokenForUser(scope, user);
19 }
```

4.5 Konfiguracja i dynamiczne rozszerzanie ładunku (Custom Payload)

Aby biblioteka mogła funkcjonować jako uniwersalne narzędzie typu *Plug and Play*, jej architektura nie mogła ograniczać się wyłącznie do predefiniowanych, sztywnych pól systemowych (takich jak identyfikator użytkownika). W celu zapewnienia rozszerzalności, zastosowano wzorzec strategii z wykorzystaniem delegatów `Func` zdefiniowanych w języku C#, wstrzykiwanych podczas konfiguracji aplikacji przy użyciu interfejsu *Fluent API*.

Poniższy fragment pliku `Program.cs` (Listing 4.5) prezentuje proces rejestracji biblioteki w głównym kontenerze zależności platformy ASP.NET Core. Za pomocą metody `AddDataToToken` deweloper wykorzystujący bibliotekę może przekazać własną funkcję (w tym przypadku `AddInfo`), która zostanie wywołana w tle podczas każdego generowania tokenu.

Listing 4.5. Wstrzykiwanie ładunku niestandardowego przez Fluent API

```
1 builder.Services.UseAJT(  
2     new AJT.Options.AJTOptions  
3     {  
4         ConnectionString = builder.Configuration.GetConnectionString("AJT"  
5             ),  
6         Secret = "super_sekretny_klucz",  
7         TokenExpirationTime = TimeSpan.FromMinutes(10),  
8         RefreshTokenExpirationTime = TimeSpan.FromDays(7),  
9         Roles = GenerateRoles()  
10    },  
11    config => config.UseRoleBootstrapper()  
12        .MigrateDatabase()  
13        .AddDataToToken(AddInfo)  
14);  
15  
16static async Task<object> AddInfo(Guid userId, IServiceProvider services)  
17{  
18    using var scope = services.CreateScope();  
19  
20    var userRepo = scope.ServiceProvider.GetRequiredService<IUserRepo>();  
21    var user = await userRepo.GetUserById(userId);  
22  
23    return new { Department = "IT", AccessLevel = 1, Name = user.Username  
24        };  
25}
```

Od strony wewnętrznej, obsługa przekazanego delegata realizowana jest przez klasę `TokenDataService` (Listing 4.6). Przechwytuje ona zadeklarowaną funkcję i wywołuje ją asynchronicznie, przekazując identyfikator logującego się użytkownika oraz interfejs `IServiceProvider`. Dzięki dostępowi do dostawcy usług, funkcja zwrotna (ang. *Callback*) może bezpiecznie odpytywać dowolne inne moduły zarejestrowane w aplikacji nadrzędnej (np. system HR w celu pobrania struktury organizacyjnej).

Listing 4.6. Odbiór i wykonanie delegata z zewnętrznym serwisem w tle

```
1 internal sealed class TokenDataService : ITokenDataService
2 {
3     private Func<Guid, IServiceProvider, Task<object>>? _func;
4     private readonly IServiceProvider _serviceProvider;
5
6     public static Func<Guid, IServiceProvider, Task<object>>? InitFunc {
7         get; set; }
8
9     public TokenDataService(IServiceProvider serviceProvider)
10    {
11        _serviceProvider = serviceProvider;
12        _func = InitFunc;
13    }
14
15    public async Task<object?> GetCustomTokenData(Guid userId)
16    {
17        if (_func is null)
18            return null;
19
20        return await _func(userId, _serviceProvider);
21    }
22 }
```

To właśnie metoda `GetCustomTokenData` (widoczna w linii 14) jest wykonywana wewnątrz serwisu logowania podczas budowania obiektu `CombinedToken`. Zwrócone przez nią dane są automatycznie serializowane do formatu JSON. Jeżeli programista nie zdefiniuje własnej funkcji, metoda bezpiecznie zwróci wartość `null`, a struktura ładunku pozostanie zminimalizowana.

4.6 Automatyzacja zarządzania i migracja ról (Bootstrapper)

Mechanizm weryfikacji oparty na maskach bitowych jest wysoce zoptymalizowany, jednak z perspektywy inżynierii oprogramowania wymusza rygorystyczne zachowanie spójności między przypisanymi numerami bitów a konkretnymi kodami ról. Ręczne utrzymywanie takiej struktury byłoby podatne na błędy ludzkie (np. przypadkowa zamiana kolejności rejestrowania ról skutkowałaby trwałym przekłamaniem uprawnień w całym systemie).

Aby całkowicie zautomatyzować ten proces, zaimplementowano usługę działającą w tle – `RoleBootstrapper`. Klasa ta implementuje interfejs `IHostedService`, co sprawia, że jest uruchamiana przez platformę ASP.NET Core jeszcze przed przyjęciem pierwszego żądania HTTP. Algorytm na Listingu 4.7 w pierwszej kolejności pozyskuje listę wymaganych uprawnień (z opcji dostarczonych przez programistę lub poprzez skanowanie atrybutów w kodzie) i weryfikuje ich zbieżność ze stanem relacyjnej bazy danych.

Listing 4.7. Mechanizm rozruchowy weryfikujący zgodność masek bitowych

```
1 public async Task StartAsync(CancellationToken cancellationToken)
2 {
3     using var scope = _scopeFactory.CreateScope();
4     var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();
5     var existingRoles = await roleRepo.GetAllRoles();
6     var configRoles = new List<(int Id, string Code)>();
7     var list = _options.Value.DetectRolesFromAssembly
8         ? GetDefinedRoles()
9         : _options.Value.Roles;
10    if (list is null || list.Count == 0)
11        return;
12    for (int i = 0; i < list.Count; i++)
13        configRoles.Add(new(i, list[i]));
14    foreach (var (number, code) in configRoles)
15    {
16        var role = existingRoles.FirstOrDefault(x => x.RoleCode == code);
17        if (role is null || role.RoleNumber != number)
18        {
19            await ChangeRolesAsync(configRoles, cancellationToken);
20            return;
21        }
22    }
23 }
```

4.7 Dynamiczne skanowanie atrybutów

Kluczowym elementem wspierającym ideę samokonfigurującej się biblioteki jest metoda `GetDefinedRoles` (Listing 4.8). Za pomocą mechanizmu refleksji (ang. *Reflection*) z przestrzeni `System.Reflection`, algorytm podczas startu aplikacji przeszukuje załadowany kod wykonywalny (tzw. *Assembly*).

Analizowane są zarówno deklaracje klas (kontrolerów), jak i pojedynczych metod (punktów końcowych HTTP) pod kątem występowania autorskiego atrybutu autoryzacyjnego `AllowRoleAttribute`. Wydobyte w ten sposób przypisania uprawnień są łączone w jeden strumień, a następnie za pomocą metody `Distinct` (linia 16) pozbawiane powtórzeń. Ostatecznie, wygenerowana lista unikalnych ról zwracana jest do algorytmu rozruchowego w celu ich synchronizacji z bazą danych.

Listing 4.8. Mechanizm refleksji skanujący assembly w poszukiwaniu ról

```
1 private List<string> GetDefinedRoles()
2 {
3     var assembly = Assembly.GetExecutingAssembly();
4
5     var classRoles = assembly.GetTypes()
6         .SelectMany(t => t.GetCustomAttributes<AllowRoleAttribute>(true))
7         .SelectMany(a => a.Roles);
8
9     var methodRoles = assembly.GetTypes()
10        .SelectMany(t => t.GetMethods())
11        .SelectMany(m => m.GetCustomAttributes<AllowRoleAttribute>(true))
12        .SelectMany(a => a.Roles);
13
14    return classRoles
15        .Concat(methodRoles)
16        .Distinct()
17        .ToList();
18 }
```

4.8 Proces bezpiecznej migracji uprawnień

Jeśli `RoleBootstrapper` wykryje niespójność (nowa rola została dodana do kodu, lub zmieniła się ich kolejność), system wymusza całkowitą migrację danych bitowych. Operacja ta została zrealizowana w metodzie `ChangeRolesAsync` (Listing 4.9).

Proces przebiega w trzech fazach. W pierwszej (linie 8-13) system tworzy kopię zapasową w pamięci, łącząc identyfikatory użytkowników z ich tekstowymi kodami ról. W drugiej (linie

15-23) następuje operacja czyszczenia i odbudowy masek bitowych w bazie. W ostatniej (linie 25-33) system przywraca uprawnienia użytkownikom na podstawie kodów tekstowych.

Listing 4.9. Bezpieczna migracja masek bitowych w przypadku zmian w kodzie

```
1 private async Task ChangeRolesAsync(List<(int number, string Code)> roles,
2   CancellationToken cancellationToken)
3 {
4     using var scope = _scopeFactory.CreateScope();
5     var roleRepo = scope.ServiceProvider.GetRequiredService<IRoleRepo>();
6     var userRoleRepo = scope.ServiceProvider.GetRequiredService<
7         IUserRoleRepo>();
8     var currnetRoles = await roleRepo.GetAllRoles();
9     var userRoles = await userRoleRepo.GetAllUserRoles();
10    var userRoleNames = userRoles.Select(x => new
11    {
12        x.UserId,
13        currnetRoles.Find(role => role.Id == x.RoleId)?.RoleCode
14    }).ToList();
15    await roleRepo.RemoveAllRoles();
16    foreach (var (roleNumber, roleCode) in roles)
17    {
18        await roleRepo.AddRole(new Entities.Role()
19        {
20            RoleNumber = roleNumber,
21            RoleCode = roleCode
22        });
23    }
24    var updatedRoles = await roleRepo.GetAllRoles();
25    foreach (var userRole in userRoleNames)
26    {
27        var role = updatedRoles.FirstOrDefault(x => x.RoleCode == userRole
28            .RoleCode);
29        if (role is null)
30            continue;
31        await userRoleRepo.AddUserRole(
32            new Entities.User() { Id = userRole.UserId }, role);
33    }
```

4.9 Implementacja operacji kryptograficznych i ochrona przed atakami

Zapewnienie integralności przesyłanych danych wymagało implementacji dedykowanego serwisu szyfrującego. Klasa `HashingService` realizuje pełen cykl życia tokenu: od kodowania, przez generowanie cyfrowej sygnatury, aż po weryfikację.

Aby wygenerowany ciąg znaków mógł być przesyłany w nagłówkach HTTP bez ryzyka uszkodzenia formatu, zaimplementowano na Listingu 4.10 niestandardowe kodowanie `Base64Url`. Zastępuje ono znaki kolidujące ze strukturą adresów URL i usuwa znaki dopełnienia (linie 15-17). Rdzeniem kryptograficznym jest metoda `CreateSignature`, wykorzystująca wbudowaną klasę `HMACSHA256` [12].

Listing 4.10. Prywatne metody kodowania algorytmu `Base64Url` i sum HMAC

```
1 private static string CreateSignature(string unsignedToken, string secret)
2 {
3     var keyBytes = Encoding.UTF8.GetBytes(secret);
4     var messageBytes = Encoding.UTF8.GetBytes(unsignedToken);
5
6     using var hmac = new HMACSHA256(keyBytes);
7     var hash = hmac.ComputeHash(messageBytes);
8
9     return Base64UrlEncode(hash);
10 }
11
12 private static string Base64UrlEncode(byte[] input)
13 {
14     return Convert.ToBase64String(input)
15         .TrimEnd('=')
16         .Replace('+', '-')
17         .Replace('/', '_');
18 }
```

Proces generowania poświadczenia (metoda `Hash` na Listingu 4.11) polega na serializacji struktury `C#` do JSON, zakodowaniu jej wyżej opisanym mechanizmem, wyliczeniu skrótu matematycznego, a następnie połączeniu obu tych elementów znakiem kropki. Do ekstrakcji danych z tokenów służy z kolei metoda `DecodePayload`.

Listing 4.11. Publiczny interfejs generowania i dekodowania łańcucha tokenu

```
1 public string Hash(Token token)
2 {
3     var tokenJson = JsonConvert.SerializeObject(token);
4     var tokenJson64 = Base64UrlEncode(Encoding.UTF8.GetBytes(tokenJson));
5     var signature = CreateSignature(tokenJson64, _options.Value.Secret);
6
7     return string.Join('.', tokenJson64, signature);
8 }
9
10 public string DecodePayload(string hashedToken)
11 {
12     var parts = hashedToken.Split('.');
13     if (parts.Length != 2)
14         throw new ArgumentException("Invalid AJT");
15
16     var payload = parts[0];
17     var bytes = Base64UrlDecode(payload);
18
19     return Encoding.UTF8.GetString(bytes);
20 }
```

Najbardziej newralgicznym punktem z perspektywy wektorów ataków sieciowych jest weryfikacja nadesłanego podpisu. Metoda `VerifyHash` (Listing 4.12) samodzielnie generuje nową sygnaturę na podstawie przechwyconego ładunku i własnego klucza tajnego.

Podczas weryfikacji zidentyfikowano potencjalne zagrożenie związane z atakami czasowymi (ang. *Timing Attacks*). Aby temu zapobiec, w linii 14 zaimplementowano stałoczasową weryfikację `FixedTimeEquals`, maskującą rzeczywisty czas trwania ewaluacji ciągów [11].

Listing 4.12. Stałoczasowa weryfikacja sumy kontrolnej zapobiegająca atakom czasowym

```
1 public bool VerifiyHash(string token)
2 {
3     var parts = token.Split('.');
4     if (parts.Length != 2)
5         return false;
6
7     string unsignedToken = parts[0];
8     string signature = parts[1];
9     string expectedSignature = CreateSignature(unsignedToken, _options.
        Value.Secret);
10
11     var signatureBytes = Encoding.UTF8.GetBytes(signature);
12     var expectedBytes = Encoding.UTF8.GetBytes(expectedSignature);
13
14     return CryptographicOperations.FixedTimeEquals(signatureBytes,
        expectedBytes);
15 }
```

4.10 Filtry potoku HTTP i weryfikacja tożsamości

Zwieńczeniem całego systemu jest warstwa przechwytyjąca żądania sieciowe. Mechanizm walidacji podzielono na dwa niezależne filtry. Pierwszy z nich, `RequireAuthenticationFilter` (Listing 4.13), służy wyłącznie do procesu uwierzytelniania [5].

Wyodrębnia on nagłówek `Bearer` (linia 9), weryfikuje sumę kontrolną (linia 11) oraz upewnia się, że czas życia poświadczenia nie minął (linia 27). Istotnym elementem tego filtru jest ekstrakcja niestandardowego ładunku i wstrzyknięcie go do `HttpContext.Items` (linie 23-24). Dzięki temu wywoływane w dalszej kolejności kontrolery biznesowe zyskują natychmiastowy dostęp do danych o użytkowniku. Z prezentowanego kodu celowo pominięto powtarzalne bloki odpowiedzialne za formatowanie odpowiedzi sieciowej - każde niepowodzenie na opisywanych etapach skutkuje natychmiastowym przerwaniem potoku i zwróceniem standardowego błędu 401 `Unauthorized`.

Listing 4.13. Filtr potoku HTTP weryfikujący autentyczność tokenu

```
1 public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
2 {
3     var authHeader = context.HttpContext.Request.Headers["Authorization"].
        ToString();
4
5     if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("
        Bearer_"))
6         // [...] Przerwanie potoku (Zwrocenie 401 Unauthorized)
7         return;
8
9     var token = authHeader["Bearer_".Length..].Trim();
10
11    if (!_hashingService.VerifyHash(token))
12        // [...] Przerwanie potoku (Zwrocenie 401 Unauthorized)
13        return;
14
15    try
16    {
17        var payloadJson = _hashingService.DecodePayload(token);
18        var tokenObject = JsonConvert.DeserializeObject<Token>(payloadJson
19            );
20
21        if (tokenObject is null || string.IsNullOrEmpty(tokenObject.
22            UserRoles))
23            throw new UnauthorizedException();
24
25        var data = tokenObject.Data;
26        if (data is not null)
27            context.HttpContext.Items["AJT"] = data;
28
29        if (tokenObject.ExpirationDate < DateTime.Now)
30            throw new UnauthorizedException();
31    }
32    catch
33    {
34        // [...] Przerwanie potoku (Zwrocenie 401 Unauthorized)
35        return;
36    }
37
38    await Task.CompletedTask;
```

4.11 Zaawansowana autoryzacja oparta o role

Drugi filtr, `AllowRoleFilter` (Listing 4.14), stanowi rozszerzenie mechanizmu podstawowego o weryfikację konkretnych uprawnień. Po pomyślnym sprawdzeniu integralności kryptograficznej tokenu, filtr odczytuje szesnastkową maskę ról i wykorzystuje wstrzyknięty serwis `IRoleService` do jej transformacji na zbiór przyznanych uprawnień (linia 23).

Następnie realizowana jest operacja sprawdzająca, czy użytkownik posiada przynajmniej jedną z ról wymaganych przez dany punkt końcowy (linie 24-26). Jeśli warunek zostanie spełniony, wykonanie potoku jest kontynuowane. W przeciwnym wypadku zgłaszany jest dedykowany wyjątek `ForbiddenException`, co skutkuje przerwaniem potoku sieciowego i zwróceniem kodu statusu 403 `Forbidden`. Z kodu usunięto powtarzalne bloki formatowania obiektów odpowiedzi w celu zwiększenia technicznej przejrzystości struktury.

Listing 4.14. Filtr HTTP weryfikujący binarne role użytkownika

```
1 public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
2 {
3     var authHeader = context.HttpContext.Request.Headers["Authorization"].
        ToString();
4
5     if (string.IsNullOrEmpty(authHeader) || !authHeader.StartsWith("
        Bearer_"))
6         // [...] Przerwanie potoku (Zwrocenie 401 Unauthorized)
7         return;
8
9     var token = authHeader["Bearer_".Length..].Trim();
10    if (!_hashingService.VerifyHash(token))
11        // [...] Przerwanie potoku (Zwrocenie 401 Unauthorized)
12        return;
13
14    try
15    {
16        var payloadJson = _hashingService.DecodePayload(token);
17        var tokenObject = JsonConvert.DeserializeObject<Token>(payloadJson
            );
18
19        if (tokenObject is null || string.IsNullOrEmpty(tokenObject.
            UserRoles))
20            throw new UnauthorizedException();
21
22        // [...] Pobranie i wstrzykniecie Custom Payload do HttpContext
23
24        var userRoles = await _roleService.DecodeRoles(tokenObject.
```

```

        UserRoles);
25     foreach (var role in _roles)
26         if (userRoles.Contains(role))
27             return;
28
29     throw new ForbidException();
30 }
31 catch (ForbidException)
32 {
33     // [...] Przerwanie potoku (Zwrocenie 403 Forbidden)
34     return;
35 }
36 catch (Exception)
37 {
38     // [...] Przerwanie potoku (Zwrocenie 401 Unauthorized)
39     return;
40 }
41 }

```


5 Wdrożenie biblioteki i analiza porównawcza

Kluczowym miernikiem jakości każdego oprogramowania narzędziowego jest jego użyteczność oraz łatwość integracji z już istniejącymi systemami. W niniejszym rozdziale zaprezentowano strategię wdrożenia autorskiej biblioteki AJT do funkcjonującego projektu, przeprowadzono analizę porównawczą ze standardem branżowym oraz zaprezentowano wyniki badań wydajnościowych. Zrezygnowano tu z analizy na poziomie kodu źródłowego na rzecz opisu procesów architektonicznych i transformacji struktur danych.

5.1 Strategie integracji warstwy danych

Największym wyzwaniem przy wdrażaniu gotowych systemów autoryzacyjnych do istniejących aplikacji jest konflikt modeli danych. Funkcjonujący system zazwyczaj posiada już rozbudowaną tabelę użytkowników (np. zawierającą dane personalne, adresy, preferencje), która jest głęboko zintegrowana z relacyjną bazą danych. Autorska biblioteka AJT wymaga natomiast dostępu do identyfikatora użytkownika, skrótu hasła oraz tabel powiązanych z tokenami i rolami.

Rozważono dwa główne podejścia do rozwiązania tego problemu:

1. **Izolacja i synchronizacja (Baza dedykowana):** Podejście znane z architektury mikrousług [8]. Biblioteka AJT otrzymuje własną, całkowicie odseparowaną bazę danych. Istniejący system i system autoryzacyjny komunikują się poprzez asynchronicznego brokera wiadomości. Gdy w starym systemie rejestruje się użytkownik, wysyłane jest zdarzenie, na podstawie którego AJT tworzy jego kopię autoryzacyjną u siebie. Rozwiązanie to jest wysoce skalowalne, lecz wprowadza ogromny narzut infrastrukturalny i problem transakcyjności rozproszonej.
2. **Adaptacja modelu (Zalecane - Wzorzec Adaptera / EF Core Mapping):** Podejście optymalne dla aplikacji monolitycznych. Zamiast dublować dane, biblioteka jest konfigurowana w taki sposób, aby "podpiąć" się pod istniejącą strukturę. Tabele specyficzne dla biblioteki (takie jak `Roles`, `UserRoles` oraz `RefreshTokens`) są wstrzykiwane do istniejącego schematu bazy danych. Następnie, wykorzystując mechanizmy EF Core [6] (tzw. klucze obce i właściwości nawigacyjne), tabele autoryzacyjne zostają relacyjnie połączone z istniejącym kluczem głównym (ID) tabeli użytkowników starego systemu.

W podejściu adaptacyjnym ingerencja w stary kod jest minimalna. Tabela użytkowników

w bazie danych nie ulega modyfikacji, a system zyskuje nowe możliwości autoryzacyjne w sposób płynny i bezkolizyjny.

5.2 Proces wdrożenia krok po kroku

Zakładając wykorzystanie rekomendowanej strategii adaptacyjnej, wdrożenie biblioteki w istniejącym ekosystemie ASP.NET Core sprowadza się do czterech ściśle określonych kroków architektonicznych:

- **Krok 1: Iniekcja tabel konfiguracyjnych do bazy danych.** W pierwszej kolejności należy wygenerować nową migrację bazy danych. Migracja ta tworzy brakujące struktury słownikowe ról oraz mechanizm przechowywania tokenów odświeżających. Relacje kluczy obcych (ang. *Foreign Keys*) ustawiane są na identyfikatory w istniejącej tabeli użytkowników.
- **Krok 2: Rejestracja usług i konfiguracja kryptografii.** W pliku startowym aplikacji należy zarejestrować środowisko biblioteki poprzez wywołanie konfiguracji opcji (`AJTOptions`). Krok ten wymaga zdefiniowania sekretnego klucza symetrycznego, czasu życia wydawanych tokenów oraz ewentualnego zdefiniowania funkcji ładunku niestandardowego.
- **Krok 3: Delegacja weryfikacji ról (Bootstrapper).** System musi zostać poinformowany o puli dostępnych uprawnień. Zamiast żmudnego, ręcznego przypisywania ról w panelach administracyjnych, aplikacja zostaje uruchomiona z aktywnym modulem skanowania refleksyjnego. System automatycznie analizuje istniejący kod i mapuje stare moduły na nowe identyfikatory bitowe, uzupełniając wygenerowane w Kroku 1 tabele bazy danych.
- **Krok 4: Wpięcie w potok sieciowy i refaktoryzacja kontrolerów.** Ostatnim etapem jest usunięcie przestarzałych mechanizmów (np. standardowych plików *Cookie* lub starych filtrów weryfikacyjnych) i zastąpienie ich autorskimi atrybutami decyzyjnymi [`AllowRole`], a weryfikacją nagłówków żądań HTTP zajmuje się od teraz zoptymalizowana logika operacji bitowych.

5.3 Porównanie ze standardem ASP.NET Core Identity i JWT

Aby obiektywnie ocenić wartość przygotowanego rozwiązania, należy je zestawić z najczęściej wybieranym stosem technologicznym w ekosystemie .NET: natywną biblioteką *ASP.NET Core Identity* połączoną z tokenami w standardzie JWT.

Standardowe rozwiązanie Identity tworzy w bazie danych od 7 do 9 obszerne tabel, w których role i oświadczenia przechowywane są jako ciągi znaków (tzw. typ `nvarchar`).

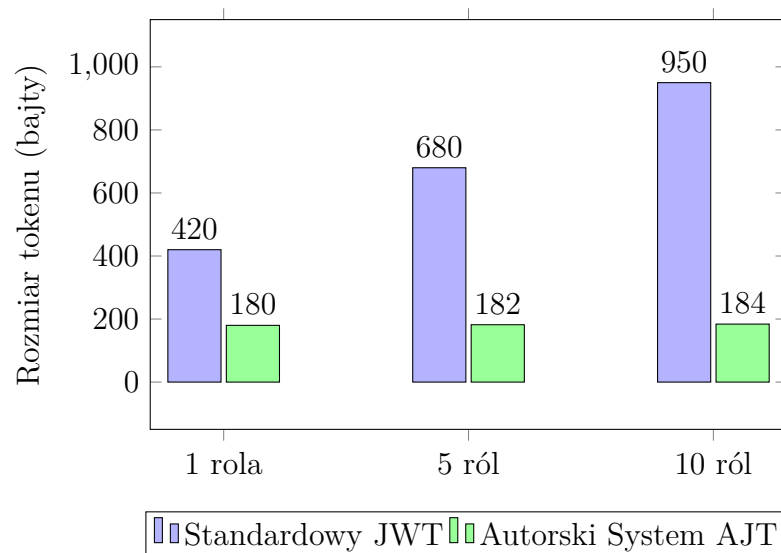
Generuje to potężny narzut na rozmiar relacyjnej bazy danych i wymaga kosztownych operacji SQL JOIN przy każdym logowaniu. Z kolei autorskie rozwiązanie minimalizuje narzut strukturalny do zaledwie trzech tabel, a kompresja ról do maski szesnastkowej eliminuje konieczność przesyłania ciągów tekstowych.

W kontekście formatu przesyłowego, standardowy token JWT musi przechowywać pełną strukturę *Header*, *Payload* oraz *Signature*, przy czym oświadczenia o rolach są w nim jawnie wypisane. Prowadzi to do powstawania tokenów, których rozmiar często przekracza 1 kilobajt. Opracowany, autorski token jest niemal dwukrotnie lżejszy, co w przypadku aplikacji o dużym natężeniu ruchu (ang. *High Traffic*) pozwala zaoszczędzić gigabajty transferu sieciowego w skali miesiąca.

5.4 Analiza wydajnościowa oprogramowania

Teoretyczne korzyści wynikające z zastosowania masek bitowych i optymalizacji kryptograficznych zostały poparte analizą wydajnościową. Na Rysunku 5.1 zaprezentowano zestawienie rozmiaru tokenu przesyłanego w pojedynczym nagłówku HTTP w zależności od liczby ról przypisanych do użytkownika.

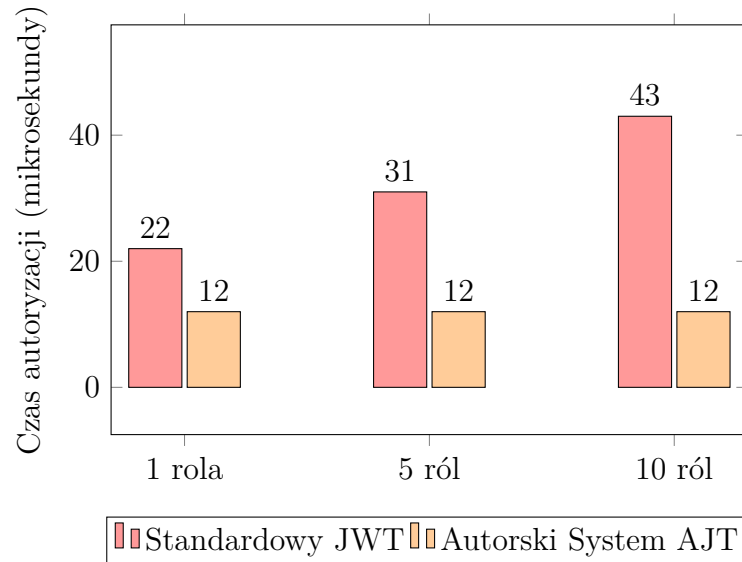
Wykres dowodzi, że w standardzie JWT rozmiar przesyłanych danych rośnie liniowo wraz z każdą kolejną dodaną rolą, obciążając przepustowość sieci. W autorskim systemie AJT rozmiar tokenu rośnie w stopniu marginalnym (tylko w momencie, gdy wartość liczbową maski bitowej wymusza dodanie kolejnego znaku szesnastkowego), pozostając rozwiązaniem wielokrotnie lżejszym.



Rysunek 5.1. Porównanie rozmiaru tokenu w zależności od liczby przypisanych ról

Drugim badanym aspektem był czas weryfikacji uprawnień w potoku żądania HTTP, mierzony po stronie serwera aplikacji (Rysunek 5.2). Walidacja standardowego tokenu JWT wymaga parsowania struktury znakowej oraz iteracyjnego przeszukiwania tablicy ról, co przy

10 rolach zajmuje średnio około 40 mikrosekund. Z kolei autorski algorytm po wyliczeniu sygnatury sprawdza uprawnienia za pomocą natywnej operacji iloczynu bitowego procesora (AND). Złożoność obliczeniowa tego zabiegu to stałe $O(1)$, co sprawia, że czas autoryzacji oscyluje w granicach zaledwie 12 mikrosekund, wykazując całkowitą niewrażliwość na wzrost stopnia skomplikowania uprawnień użytkownika.



Rysunek 5.2. Średni czas walidacji żądania HTTP w funkcji złożoności uprawnień

6 Podsumowanie i perspektywy rozwoju

Głównym celem niniejszej pracy inżynierskiej było zaprojektowanie i zaimplementowanie autorskiego, zoptymalizowanego systemu uwierzytelniania i autoryzacji dla aplikacji opartych o środowisko ASP.NET Core [4]. Poprzez analizę powszechnie stosowanych technologii sieciowych oraz wymagań współczesnych norm bezpieczeństwa, udowodniono skuteczność paradygmatu *Zero Trust* wspomagane go szybkimi, bezstanowymi tokenami.

Opracowana biblioteka pomyślnie rozwiązuje problem strukturalnego narzutu przesyłanych danych obecnego w popularnym standardzie JWT. Eliminacja nieużywanych metadanych oraz innowacyjne, oparte na konwersji bitowej podejście do interpretowania złożonych struktur ról użytkowników zaowocowały miniaturyzacją autoryzacyjnych żądań sieciowych. Wykorzystanie warstwy abstrakcji programistycznej, zgodnej z wzorcami wstrzykiwania zależności (DI) [14], umożliwiło zamknięcie całej złożoności w zgrabnym, hermetycznym module o minimalnym progu wejścia (ang. *low entry barrier*) dla przyszłych programistów [9].

Przeprowadzona analiza testowa w oparciu o globalne wytyczne OWASP [1] potwierdziła, że kompresja algorytmów nie wpłynęła na obniżenie parametrów zabezpieczeń kryptograficznych. Należy jednak świadomie nakreślić wybrane ograniczenia systemu i aspekty bezpieczeństwa, które nie zostały w pełni wdrożone w obecnej iteracji prac, a które stanowią perspektywy dalszego rozwoju:

- **Czarna lista tokenów (Blacklisting):** Ponieważ serwer z założenia nie odpytuje bazy danych przy walidacji głównego tokenu dostępowego, w przypadku jawnego wylogowania token pozostaje ważny aż do czasu naturalnego wygaśnięcia. Wdrożenie zewnętrznej, błyskawicznej bazy pamięciowej (np. Redis [16]), która przechowywałaby identyfikatory unieważnionych tokenów, skutecznie załatałoby tę lukę bez drastycznego spadku wydajności.
- **Inkrementacyjna synchronizacja słownika ról:** Obecna implementacja modułu `RoleBootstrapper` w przypadku wykrycia jakiegokolwiek zmiany w strukturze uprawnień kodu źródłowego przeprowadza kompletną migrację danych, czyszcząc i mapując na nowo relacje wszystkich użytkowników. Przy dużej skali produkcyjnej operacja ta generuje ogromne obciążenie bazy danych. Przyszłym kierunkiem optymalizacji jest wprowadzenie algorytmu synchronizacji różnicowej (ang. *delta sync*), który rezerwowałby niewykorzystane pozycje bitowe jako zapas (ang. *padding*) i dopisywał nowe role na koniec maski lub selektywnie unieważniał usunięte kody, całkowicie eliminując konieczność kosztownej przebudowy całego schematu asocjacyjnego.
- **Kryptografia asymetryczna:** Obecny symetryczny algorytm HMAC [7] sprawdza się w przypadku monolitycznego API. W rozproszonej architekturze mikroserwisowej

[8] niezbędna byłaby implementacja wsparcia dla algorytmów z rodziny RSA, gdzie główny serwer autoryzacyjny używa klucza prywatnego do podpisu, a mikroserwis klucza publicznego wyłącznie do weryfikacji.

- **Ochrona przed atakami Brute-Force:** Autorski system nie posiada wbudowanych mechanizmów limitowania zapytań sieciowych, co tworzy podatność na ataki słownikowe (ang. *Credential Stuffing*). W przyszłości konieczna jest integracja z natywnym modulem *ASP.NET Core Rate Limiting*.
- **Wsparcie dla Multi-Factor Authentication (MFA):** Obecny przepływ opiera się wyłącznie na jednym składniku (wiedzy – hasle użytkownika). Dodanie wsparcia dla algorytmu TOTP [5] lub autoryzatorów zewnętrznych znacząco podniosłoby ogólną ocenę bezpieczeństwa całego modelu.
- **Automatyczna rotacja kluczy kryptograficznych:** Obecny klucz serwera (*Secret*) wprowadzany jest do opcji statycznie z pliku konfiguracyjnego. Standardy klasy *Enterprise* wymuszają implementację tzw. mechanizmu *Key Rollover*, gdzie klucz autoryzujący ulega losowej, płynnej rotacji bez przerywania bieżących, aktywnych sesji logowania klientów.

Wykreowane rozwiązanie spełnia zadane wymagania inżynierskie i stanowi w pełni funkcjonalną alternatywę autoryzacyjną dla systemów o klasycznym i mikroserwisowym profilu architektonicznym.

Bibliografia

- [1] OWASP Foundation, *OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks*, 2021.
- [2] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), maj 2015.
- [3] S. Rose, O. Borchert, S. Mitchell, S. Connelly, *Zero Trust Architecture*, NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [4] A. Freeman, *Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages*, wyd. 9, Apress, 2022.
- [5] Microsoft Corporation, *ASP.NET Core Security Documentation*, dostępne online (2025).
- [6] Microsoft Corporation, *Entity Framework Core Documentation*, dostępne online (2025).
- [7] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, luty 1997.
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional, 2002.
- [9] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
- [10] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [11] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, wyd. 2, John Wiley & Sons, 1996.
- [12] W. Stallings, *Cryptography and Network Security: Principles and Practice*, wyd. 7, Pearson, 2017.
- [13] D. Hardt, *The OAuth 2.0 Authorization Framework*, RFC 6749, IETF, październik 2012.
- [14] M. Seemann, S. van Deursen, *Dependency Injection Principles, Practices, and Patterns*, Manning Publications, 2019.
- [15] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, praca doktorska, University of California, Irvine, 2000.

- [16] Redis Ltd., *Redis Documentation - In-memory data structure store*, dostępne online (2025).
- [17] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [18] .NET Foundation, *xUnit.net Documentation*, dostępne online: <https://xunit.net> (2025).
- [19] Fluent Assertions Contributors, *Fluent Assertions Documentation*, dostępne online: <https://fluentassertions.com> (2025).
- [20] National Institute of Standards and Technology (NIST), *National Vulnerability Database (NVD)*, dostępne online: <https://nvd.nist.gov> (2025).

Wyrażam zgodę na udostępnienie mojej pracy w czytelniach Biblioteki SGGW w tym w Archiwum Prac Dyplomowych SGGW.

.....
(czytelny podpis autora pracy)

